EL KHAZROUNI HICHAM
44630

TIPE

Objectif :

Suivre un objet en mouvement avec une caméra

Objectif :
Suivre un objet en mouvement avec une caméra

Comment suivre un objet de forme complexe en mouvement dans un environnement non contrôlé à l'aide d'une caméra mobile pilotée sur un axe de translation ?

Sommaire

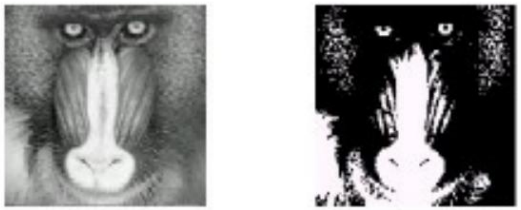
Détection de l'objet

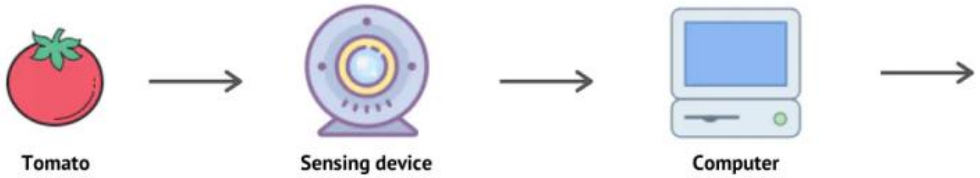
- Principe de la vision ordinateur
- Programmes de détection
- Filtres et Intelligence artificielle

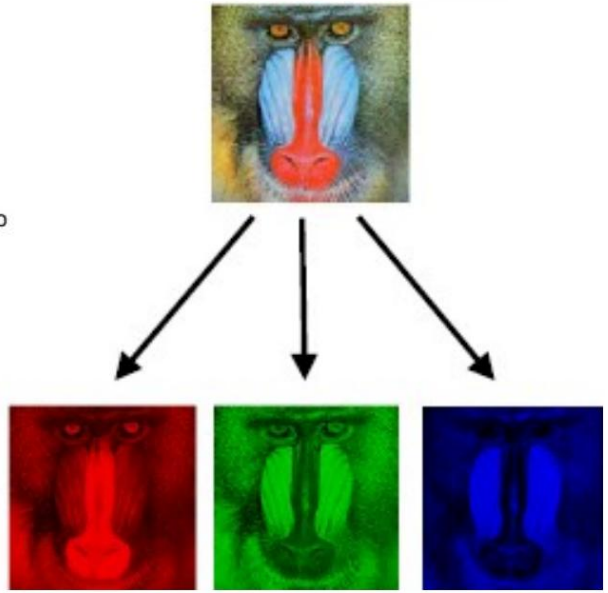
Suivi de l'objet

- Slider
- Asservissement
  - Chaine d'énergie et d'information
- Stabilisateur

Comment un ordinateur peut voir ?

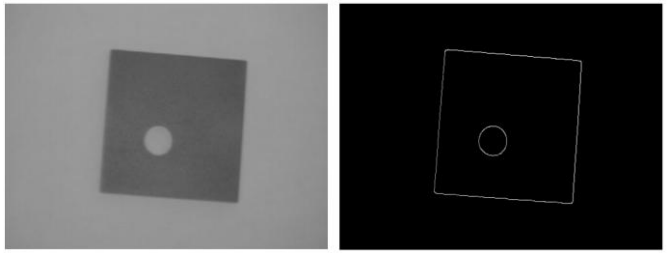




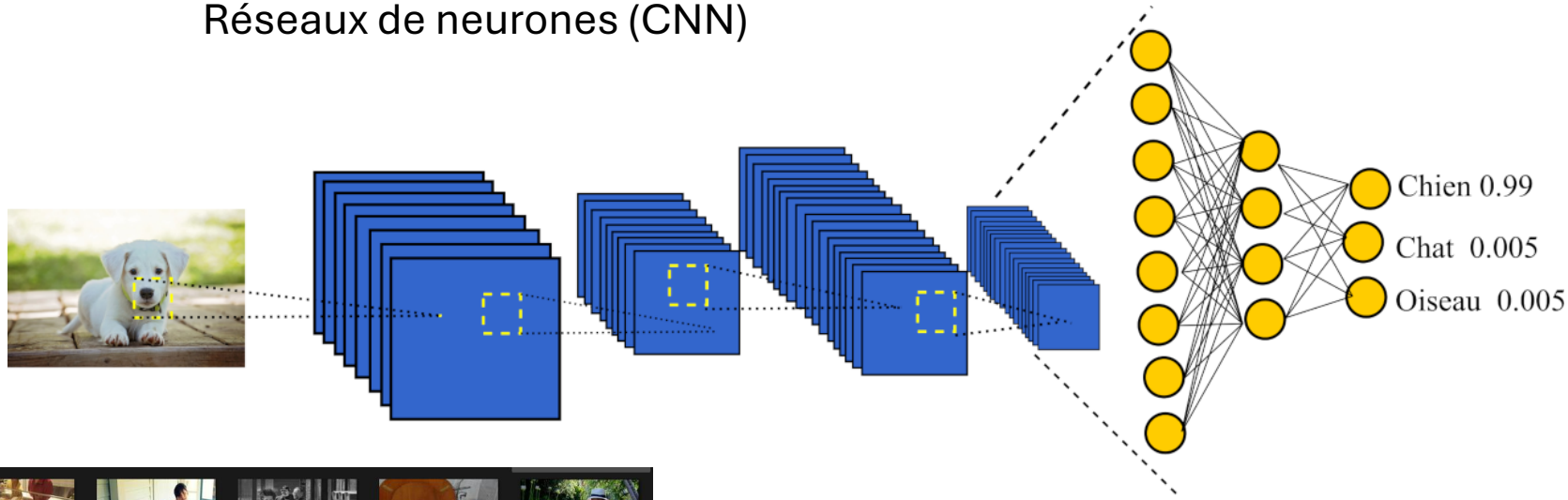


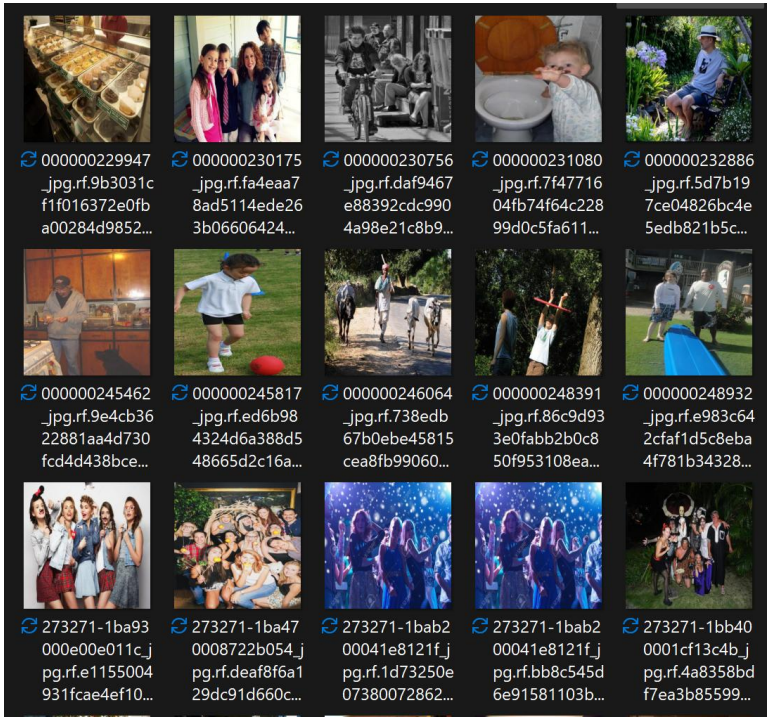
Méthodes classiques

→ Filtres



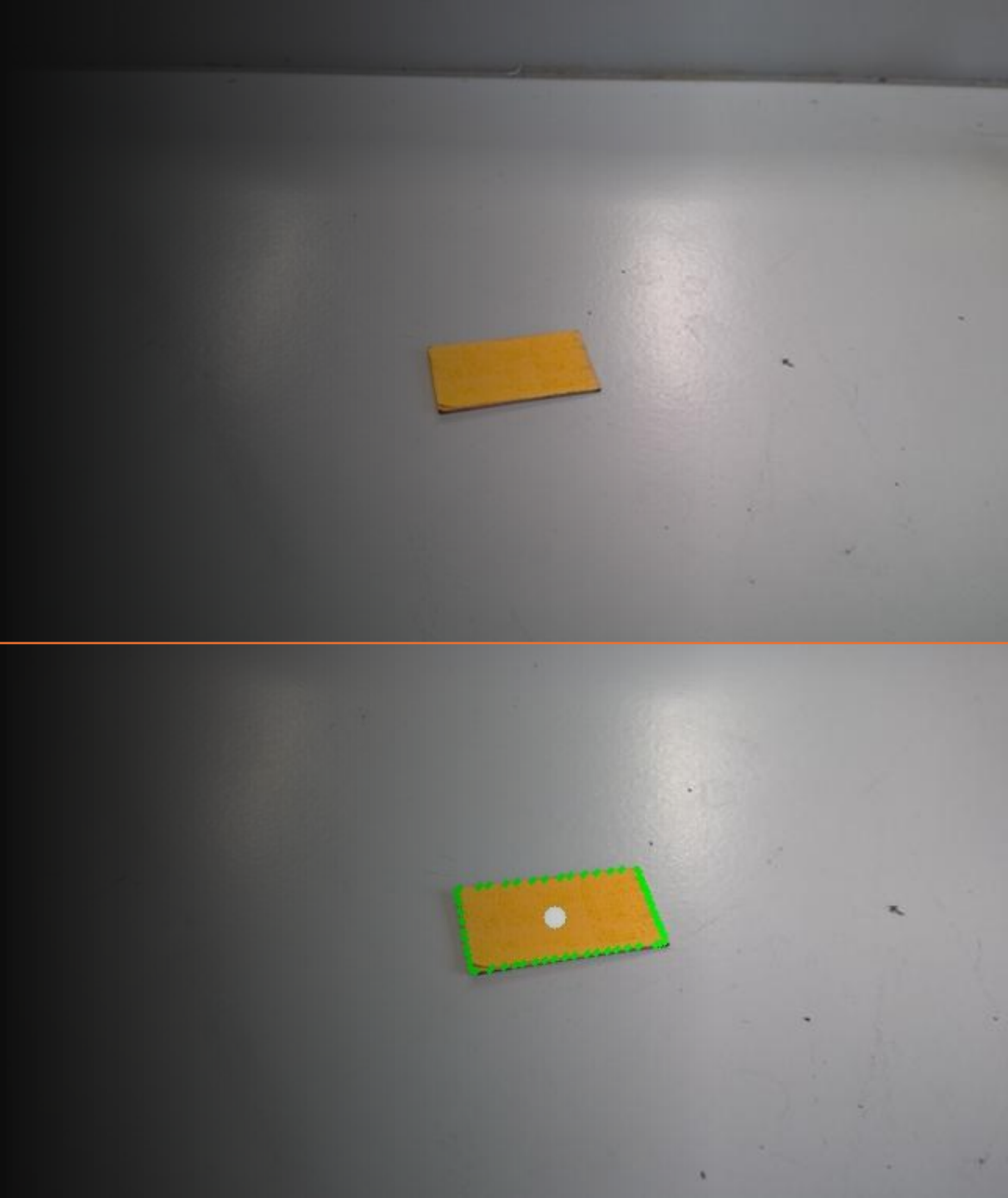
Méthode récente avec l'intelligence artificielle
Réseaux de neurones (CNN)

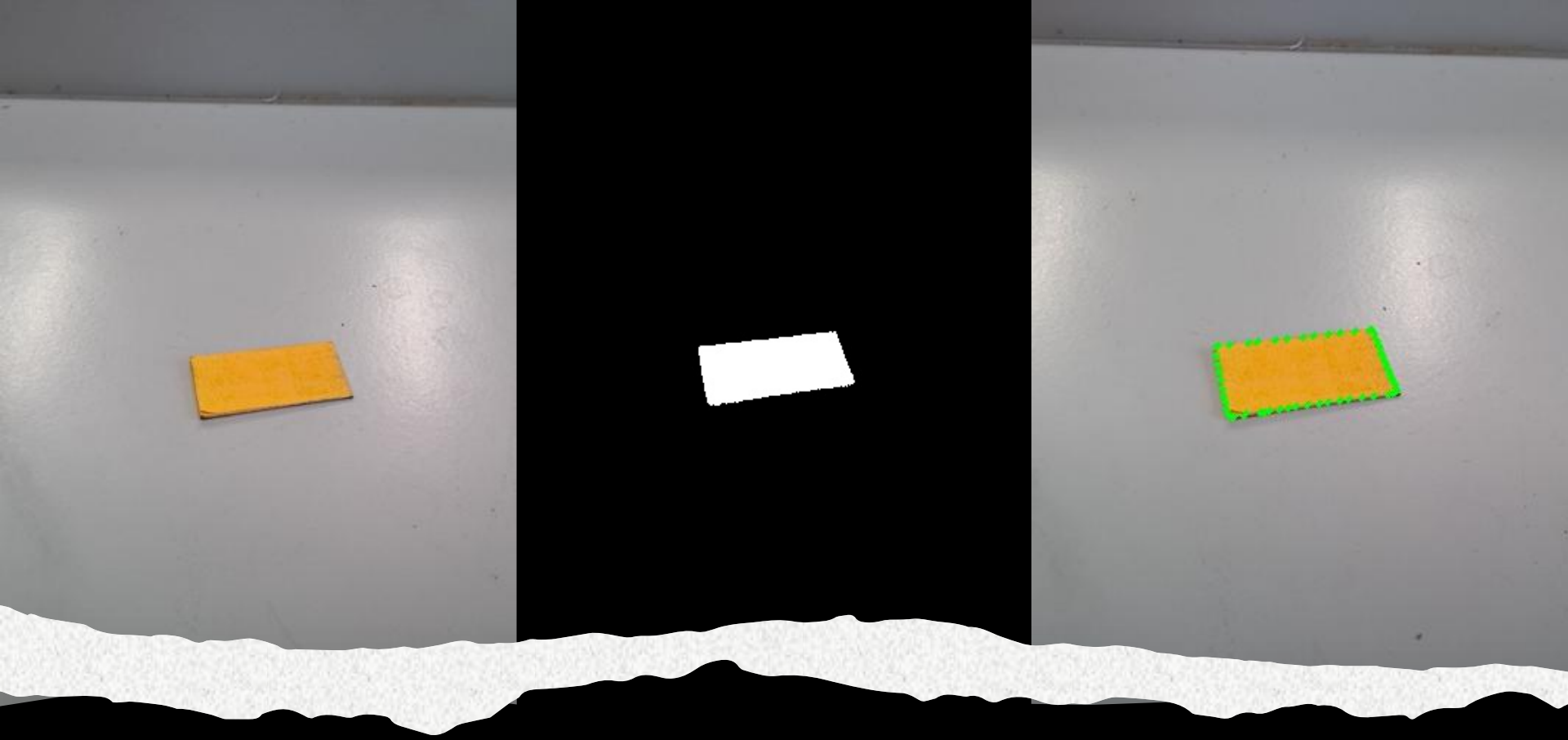




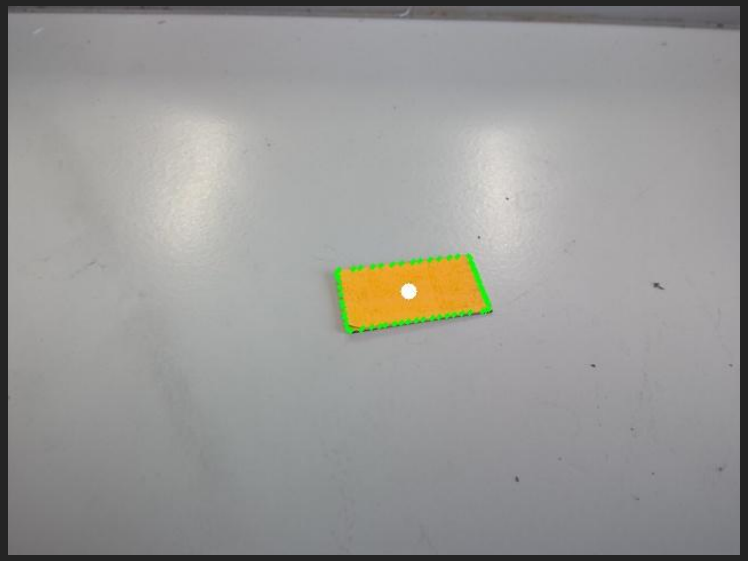
Exemples de données fournies au CNN
banque de 1687 images (roboflow)

Principe du suivi d’objet

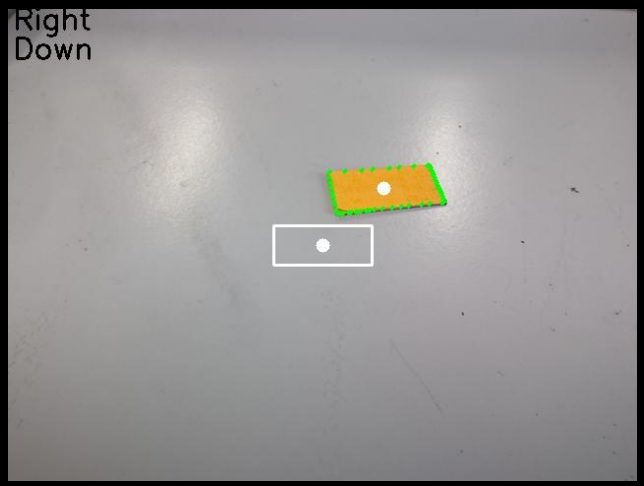




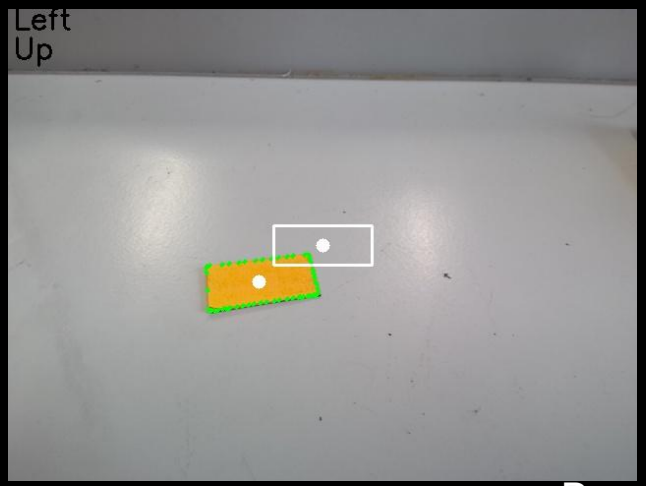
On applique un filtre à notre image : extraction des informations importantes



On récupère le centre de l’objet

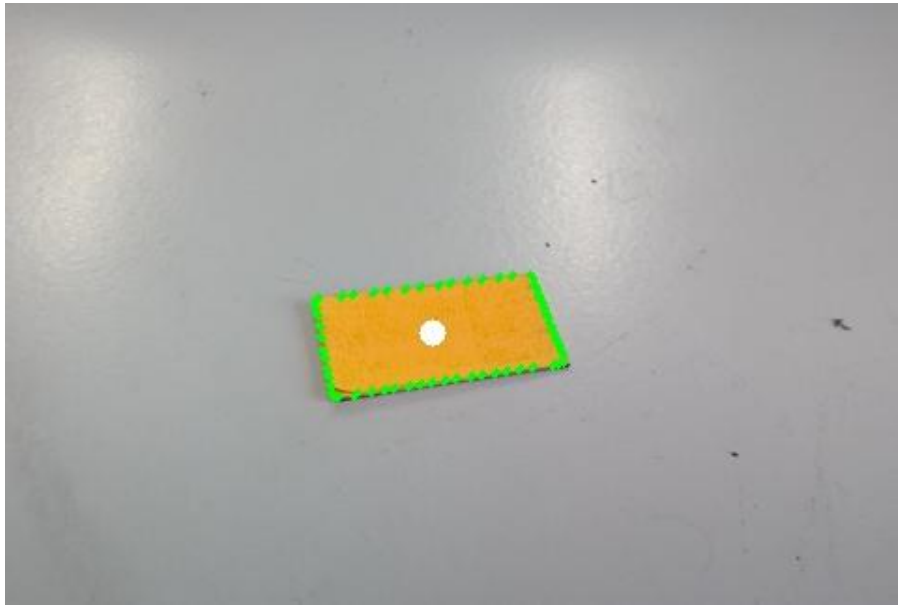


On compare la position de l’objet avec le centre de l’image



Programme de détection d’objet

1ème version



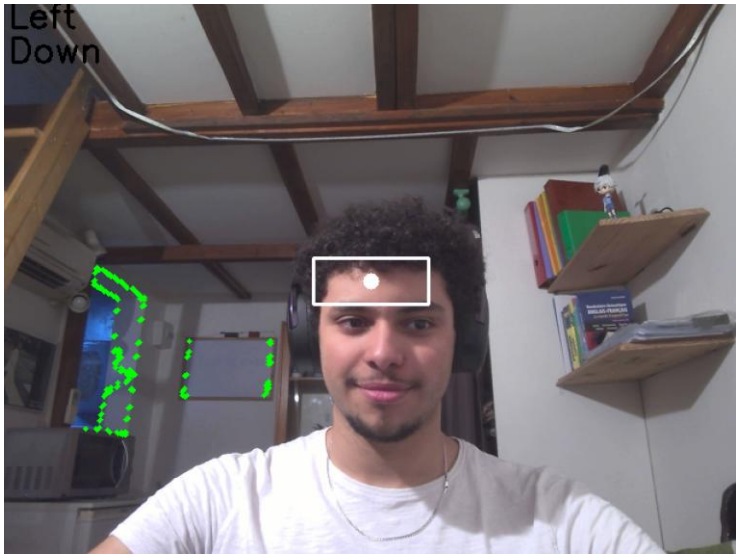
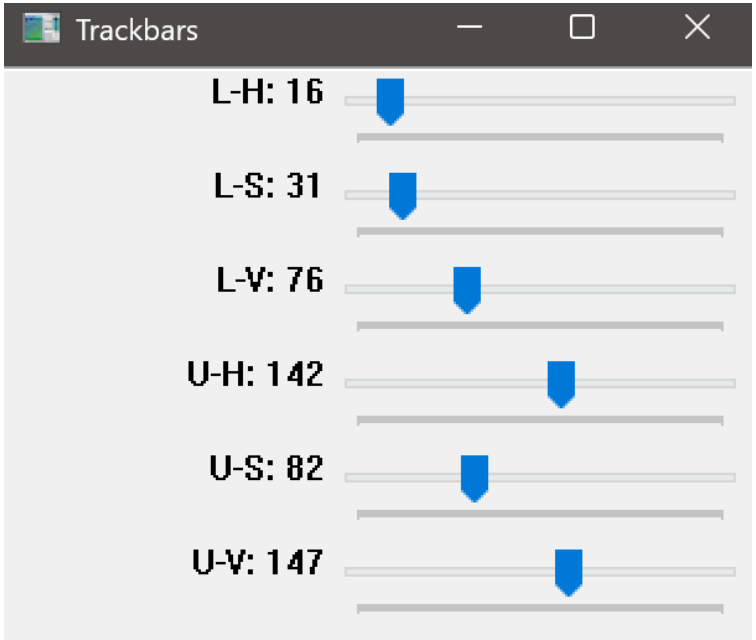


Image à traiter



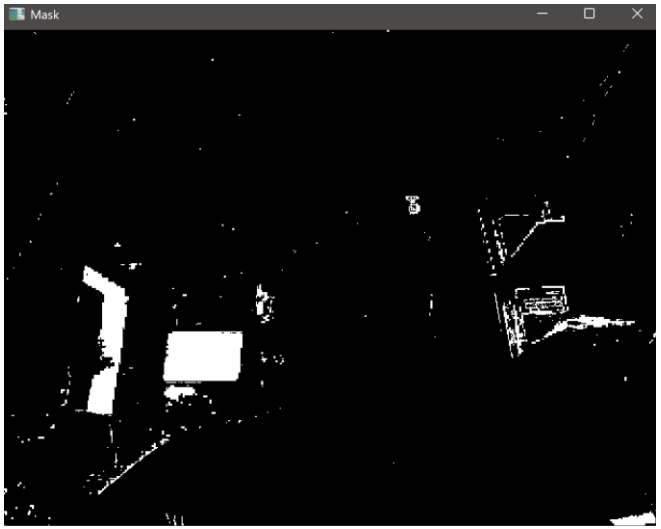


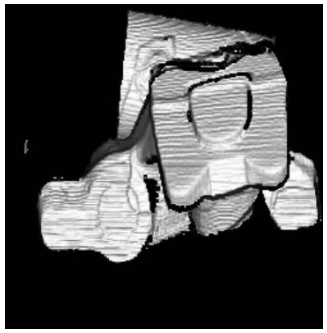
Image obtenue avec le filtre

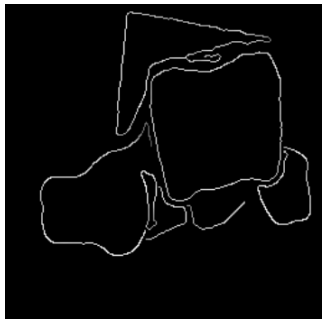
```
colors = {"blue": [94, 80, 2, 126, 255, 255],
          "beige": [0, 64, 21, 116, 205, 128],
          "white": [0, 0, 255, 73, 101, 255],
          "pink": [126, 179, 167, 193, 255, 255],
          "yellow":  [0, 116, 206, 33, 198, 255],
          "light_green":  [31, 31, 172, 68, 198, 255],
          "orange": [7, 134, 54, 69, 255, 255],
          "black": [0, 0, 0, 255, 255, 10],
          "green": [20, 95, 36, 94, 154, 186],
          "red": [2, 146, 76, 255, 255, 255]
         }
```

```
"blue": [94, 80, 2, 126, 255, 255],
```

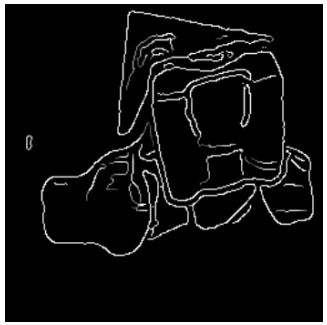
Seuil bas

Seuil haut

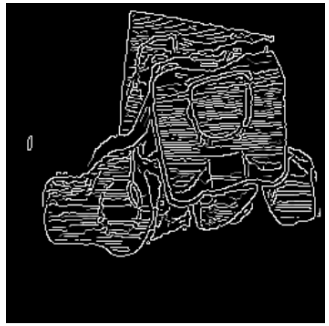




Trop sélectif



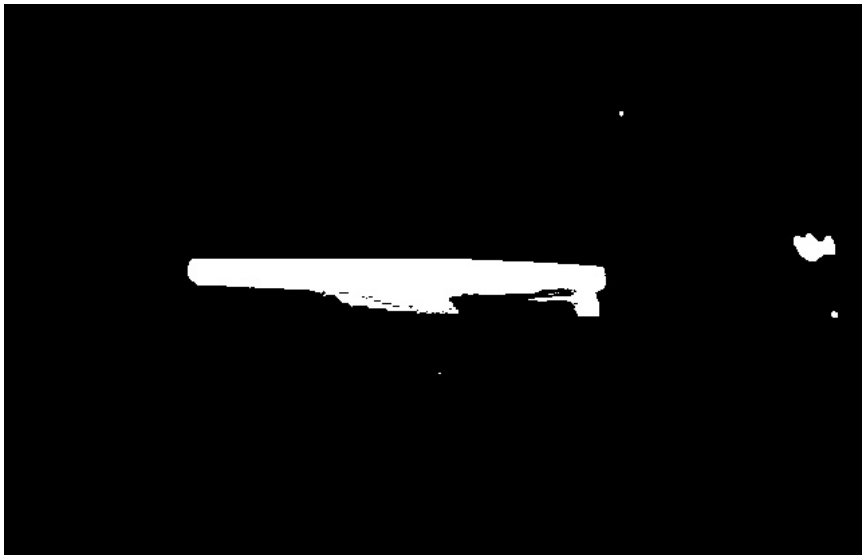
Assez sélectif



Peu sélectif

Programme de suivi d’objet

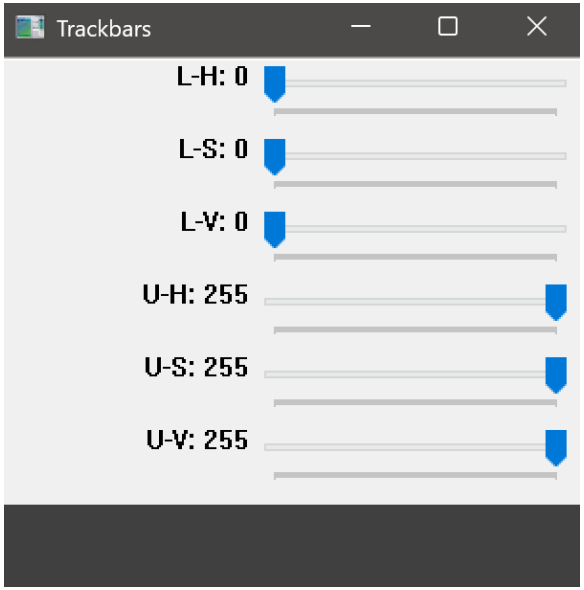
2ème version

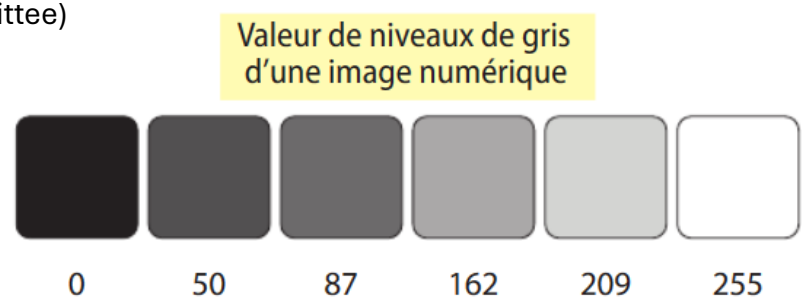


Principe

Grey = 0.299 · Red + 0.587 · Green + 0.114 · Blue

norme NTSC (National Television Standard Committee)

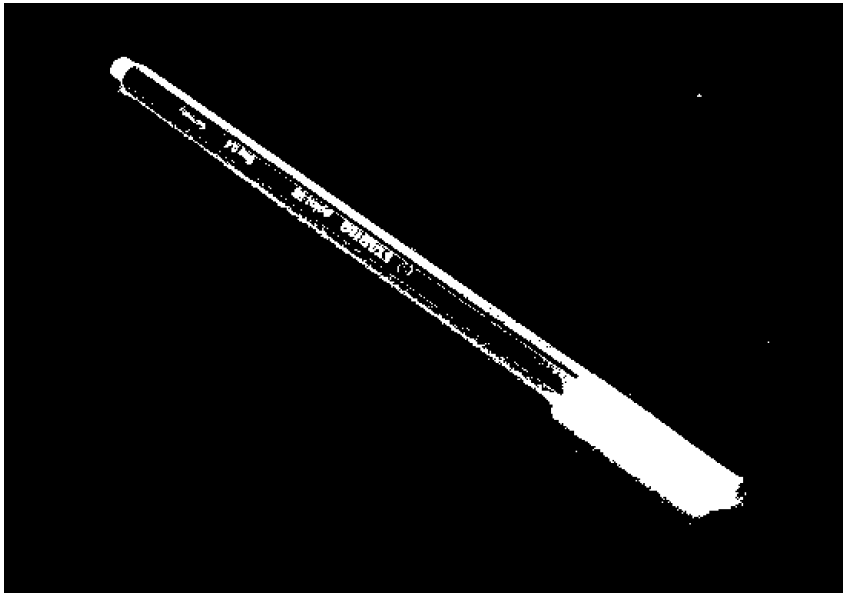





```
l = 120
h = 255
_, mask = cv2.threshold(grey_frame, l, h, cv2.ADAPTIVE_THRESH_GAUSSIAN_C)
mask = cv2.inRange(grey_frame, l, h)
```

Programme Filtre “from scratch”

3ème version

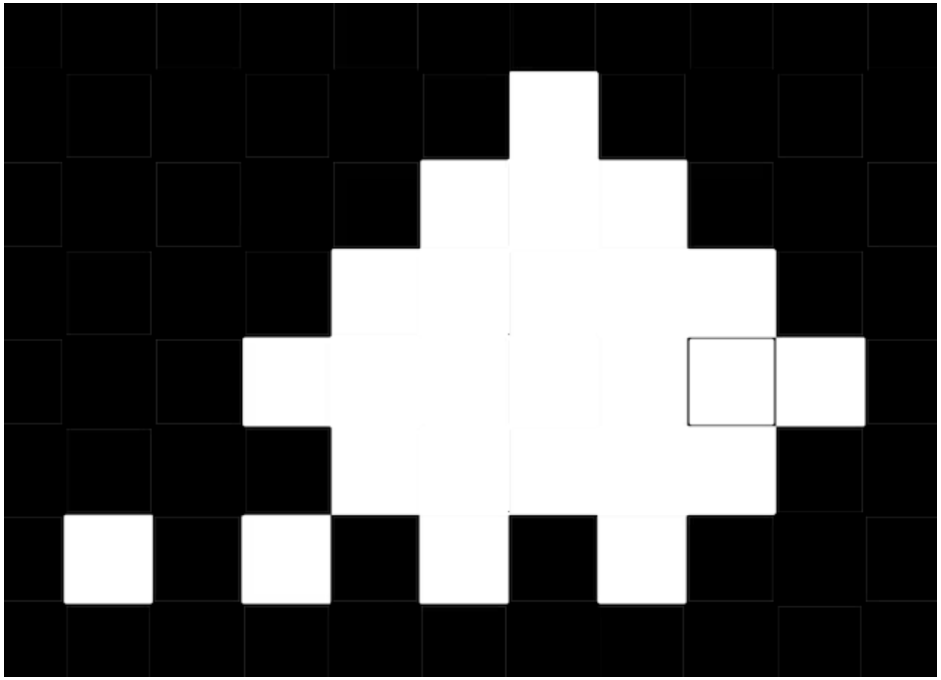


Principe

Grey = 0.299 · Red + 0.587 · Green + 0.114 · Blue

norme NTSC (National Television Standard Committee)

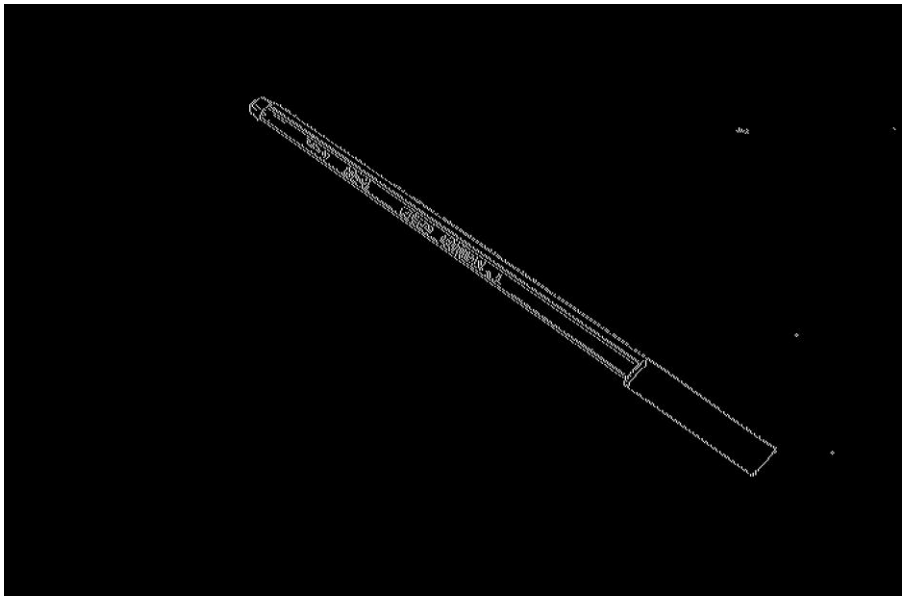
$$G = \left(\frac{1}{N}\sum_{1}^{N} x_i, \frac{1}{N}\sum_{1}^{N} y_i\right)$$



Complexité : $O(n^2)$

Programme Filtre de Canny

4ème version



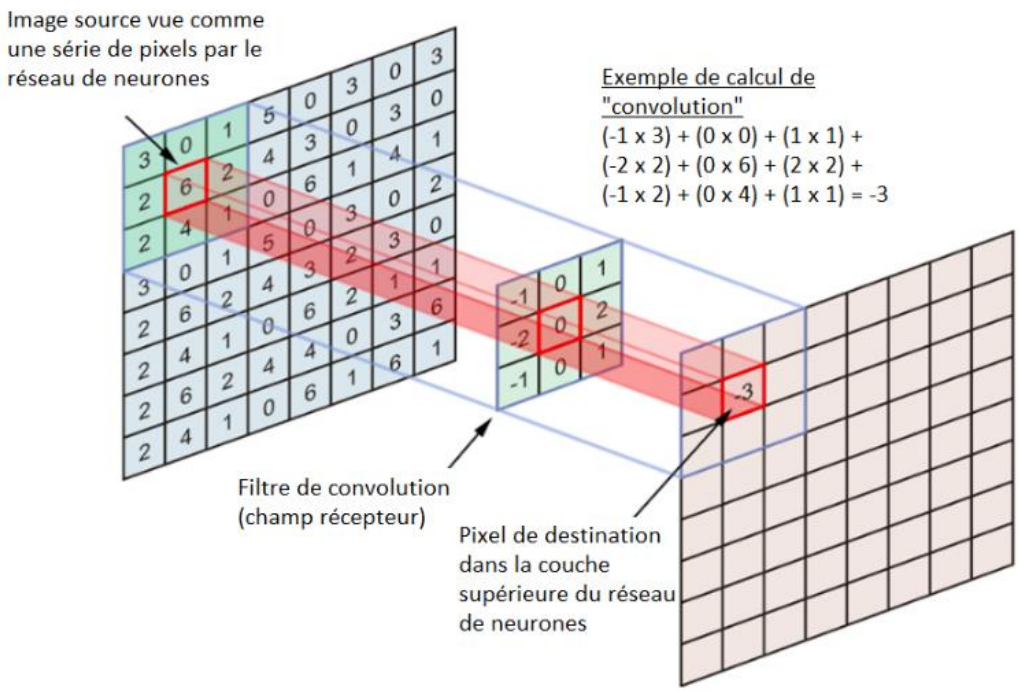
Filtre de Canny (1)

1. Flou gaussien
2. Approximation du gradient d'intensité par convolution

$$\nabla I = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y}\right)$$

$$S_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

$$S_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

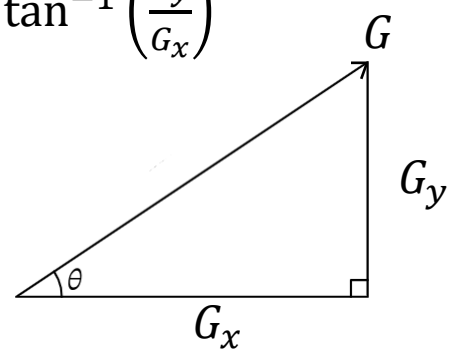


Filtre de Canny (2)

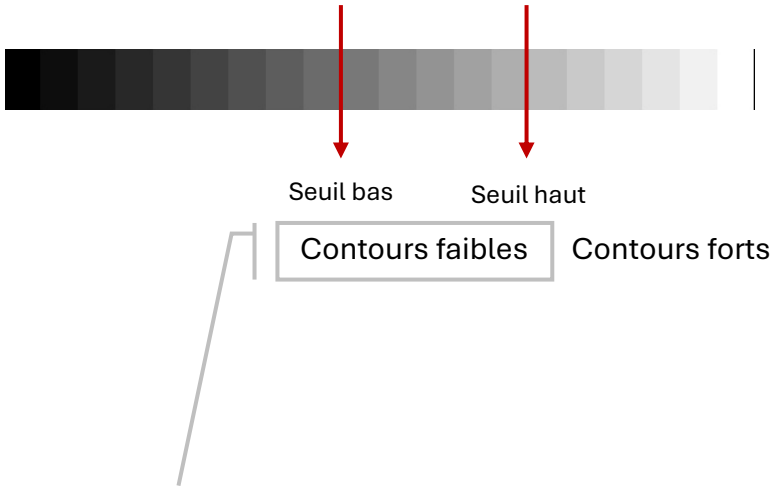
Gradient élevé → contours

Suppression des non-maxima

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$



Seuillage par hystérésis



Sélection si entouré de contours forts ou non

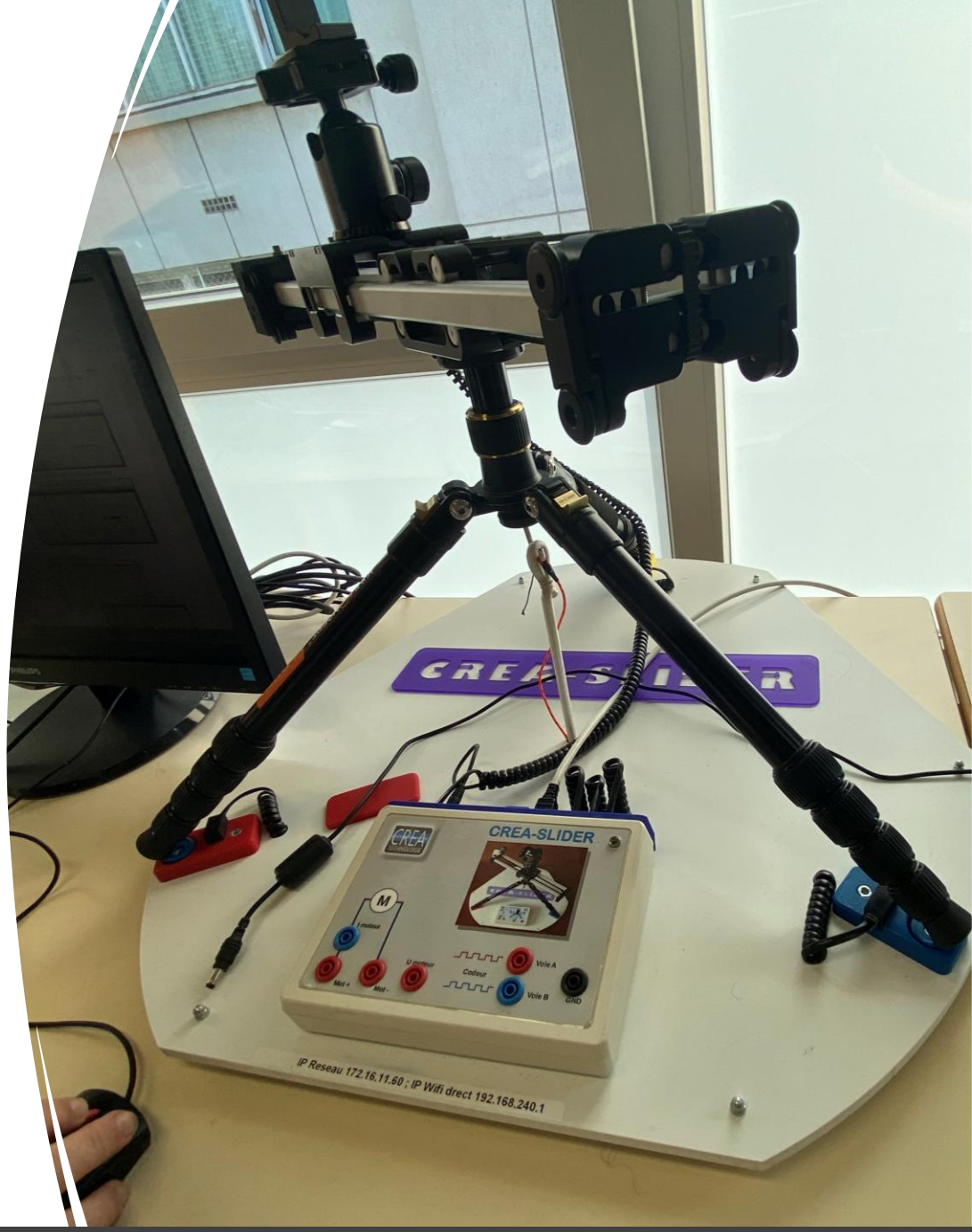
Comparatif filtre/IA

Critère	Filtres classiques	Intelligence Artificielle
Avantages	Simples à implémenter	Très grande précision sur tâches complexes
	Interprétables (chaque étape est comprise)	Capacité d'apprentissage automatique
	Peu coûteux en calcul	Robuste aux variations (bruit, lumière)
	Performants sur tâches simples	Adaptable à grande échelle

Critère	Filtres classiques	Intelligence Artificielle
Inconvénients	Sensibles au bruit et aux variations	Boîte noire (peu interprétable)
	Peu performants pour tâches complexes	Besoins en données importants
	Réglages manuels nécessaires	Coût de calcul élevé
		Temps d'entraînement long

Le Slider

CREA-SLIDER


CREA-SLIDER
Moteur
Codeur
Voie A
Voie B
GND
IP Reseau 172.16.11.60 ; IP Wifi drect 192.168.240.1

Contrôle du slider

NOUVEAUTÉ 2018

CREA SLIDER

SYSTÈME AVEC 2 AXES MOTORISÉS ASSERVIS

SI

GPGE

Système de travelling vidéo compact professionnel haut de gamme.

Pilotage et acquisition sur navigateur Web, sans aucune installation.

- CINÉMATIQUE ORIGINALE permettant un débattement égal au double de la longueur.
- TREPIED autorisant un positionnement quelconque du rail en position horizontale, inclinée ou verticale.
- PLUSIEURS MOTORÉDUCTEURS pour un apprentissage du dimensionnement à partir du modèle.
- MESURE DES GRANDEURS PHYSIQUES : tension, intensité, position, vitesse, accélération et efforts entre le sol et les trois pieds. Position de la tête et distance du sujet.
- ÉTUDE DE L'ASSERVISSEMENT en vitesse et en position du chariot, suivi de trajectoire de la tête.
- PROGRAMMATION vous pouvez prendre la commande de votre slider en Python ou Labview avec un câble USB et entrer vote propre loi de commande à partir de toutes les mesures accessibles.

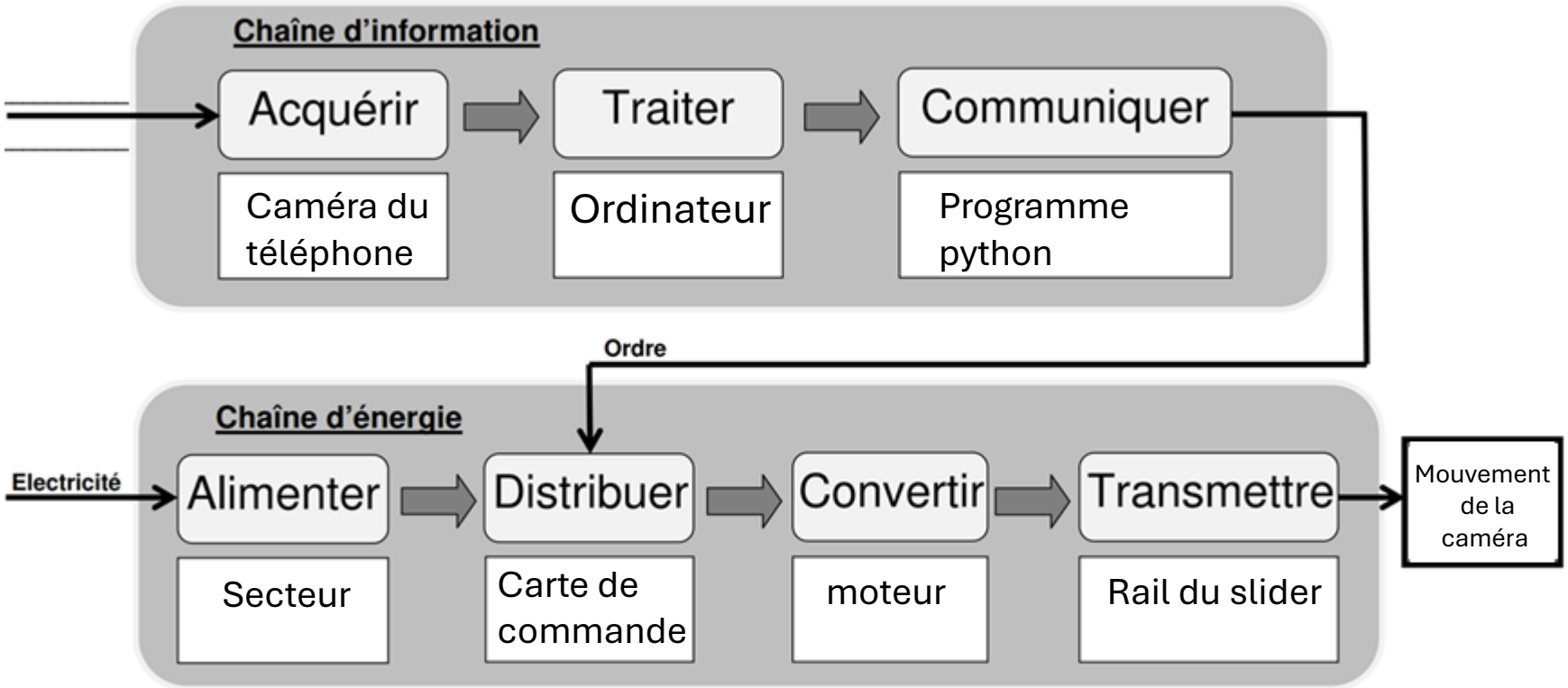
- CONTEXTE D'UTILISATION : équipé d'une caméra ou d'un smartphone, ce système permet des travelling linéaires et des prises de vues rotatives lorsque vous utilisez le mode "focus" .
- MISE EN OEUVRE : Vissez le rail sur son pied, posez le pied sur les capteurs de force, branchez le bloc secteur et connectez vous en WIFI ou en Ethernet. Le serveur Web est inclus sur la carte d'acquisition, ouvrez votre navigateur internet et le slider est opérationnel.
- MISE A JOUR : le slider est un nouveau produit, il va évoluer rapidement, vous le connectez à votre réseau et d'une simple commande, la mise à jour est complète.

Accueil sur navigateur

Accueil sur navigateur

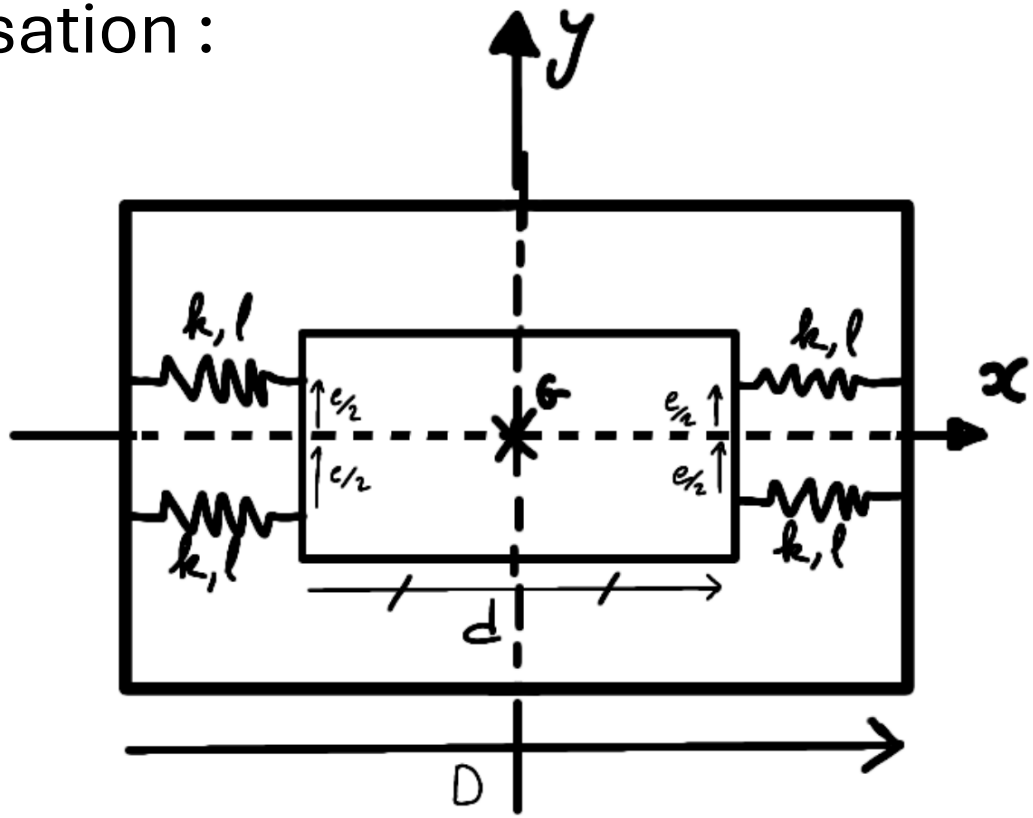
PROGRAMMATION vous pouvez prendre la commande de votre slider en Python ou Labview avec un câble USB et entrer vote propre loi de commande à partir de toutes les mesures accessibles.

Chaine fonctionnelle

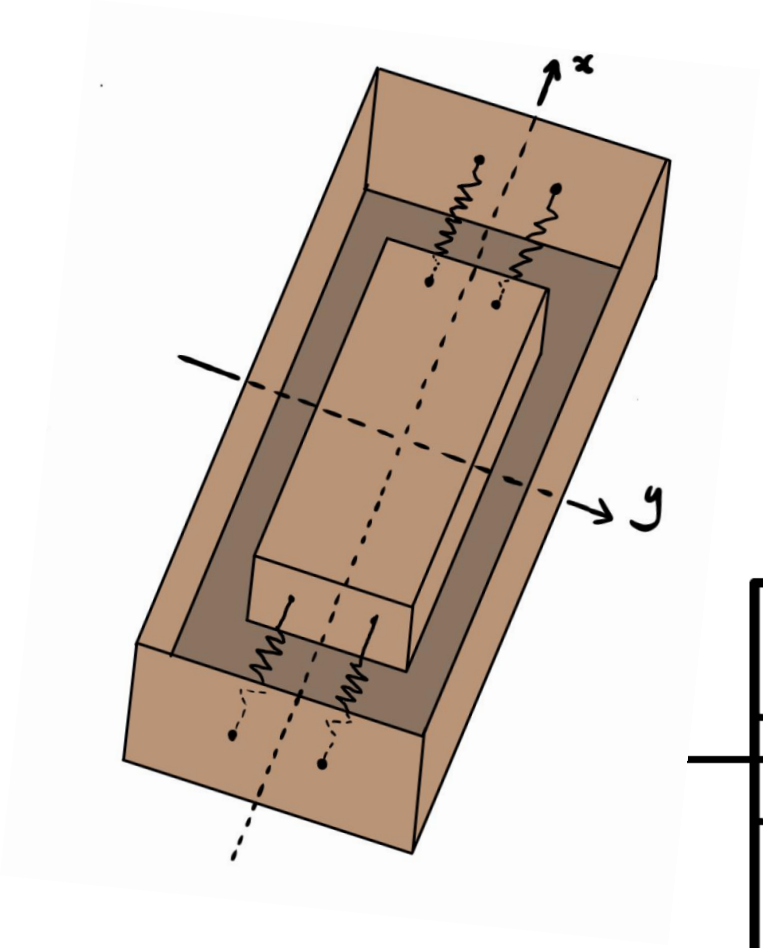


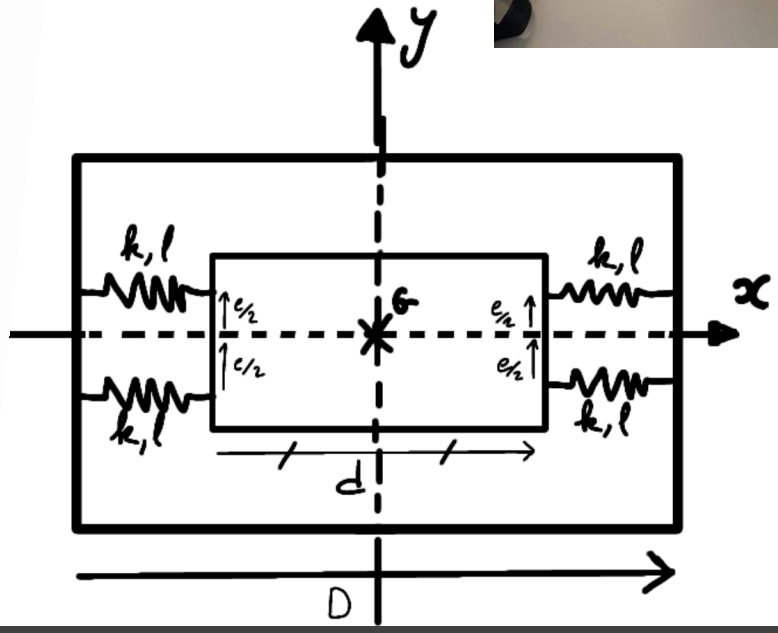
Un problème : les à-coups
Une idée : un stabilisateur

- Modélisation :

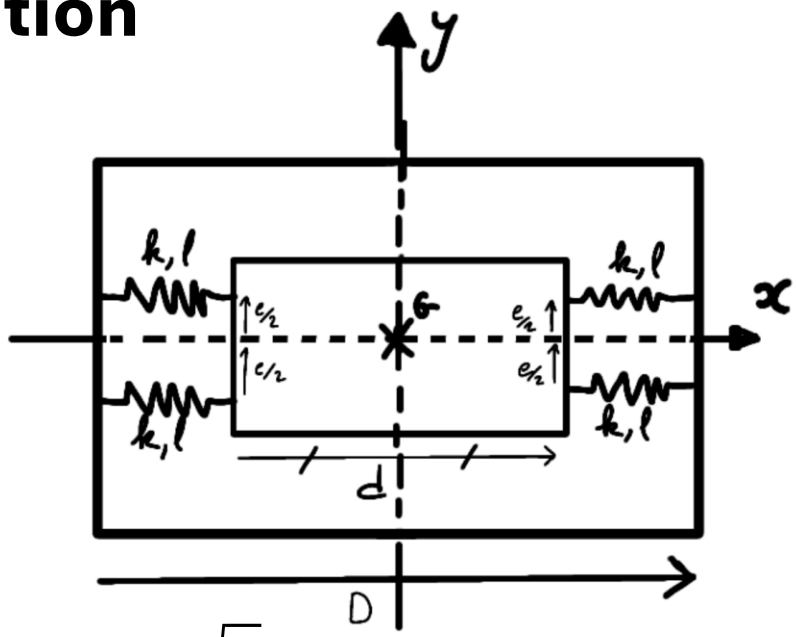


Conception du stabilisateur





Mise en équation



$$m\,\ddot{x} + 4kx + \varepsilon F_f = 0$$

avec $\varepsilon = \begin{cases} 1 \; si \; \dot{x} > 0 \\ -1 \; si \; \dot{x} < 0 \end{cases}$

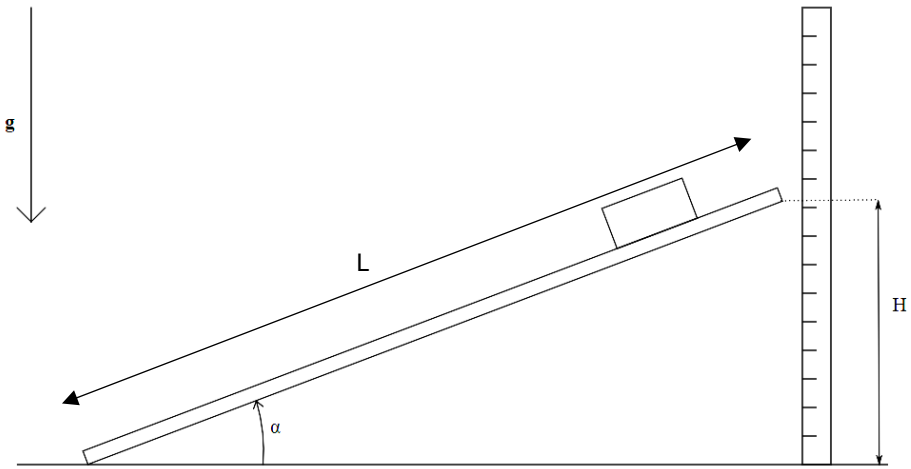
Coulomb → $F_f = \left|\vec{T}\right| = f\left|\vec{N}\right| = fmg$

Donc $$\ddot{x} + \omega_0^2\, x + \varepsilon f g = 0 \;\text{ avec }\; \omega_0 = 2\sqrt{\frac{k}{m}}$$

Résolution de l'équation par étapes

$$x_n = (-1)^n \left(x_0 - 2n\frac{fg}{\omega_0^2}\right)$$

Détermination coefficient de frottements



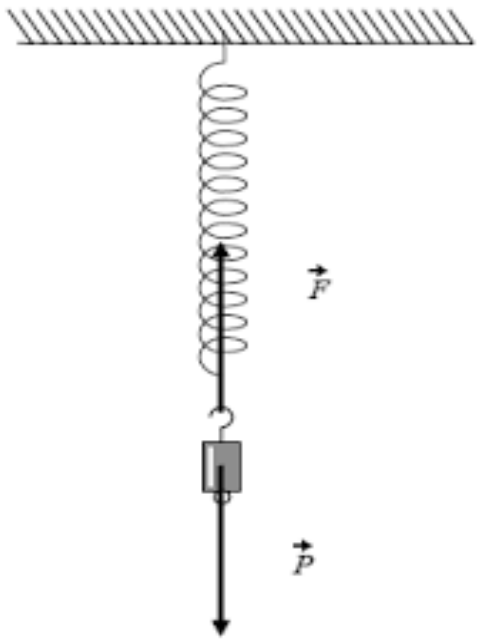
PFD sur le système {objet posé}

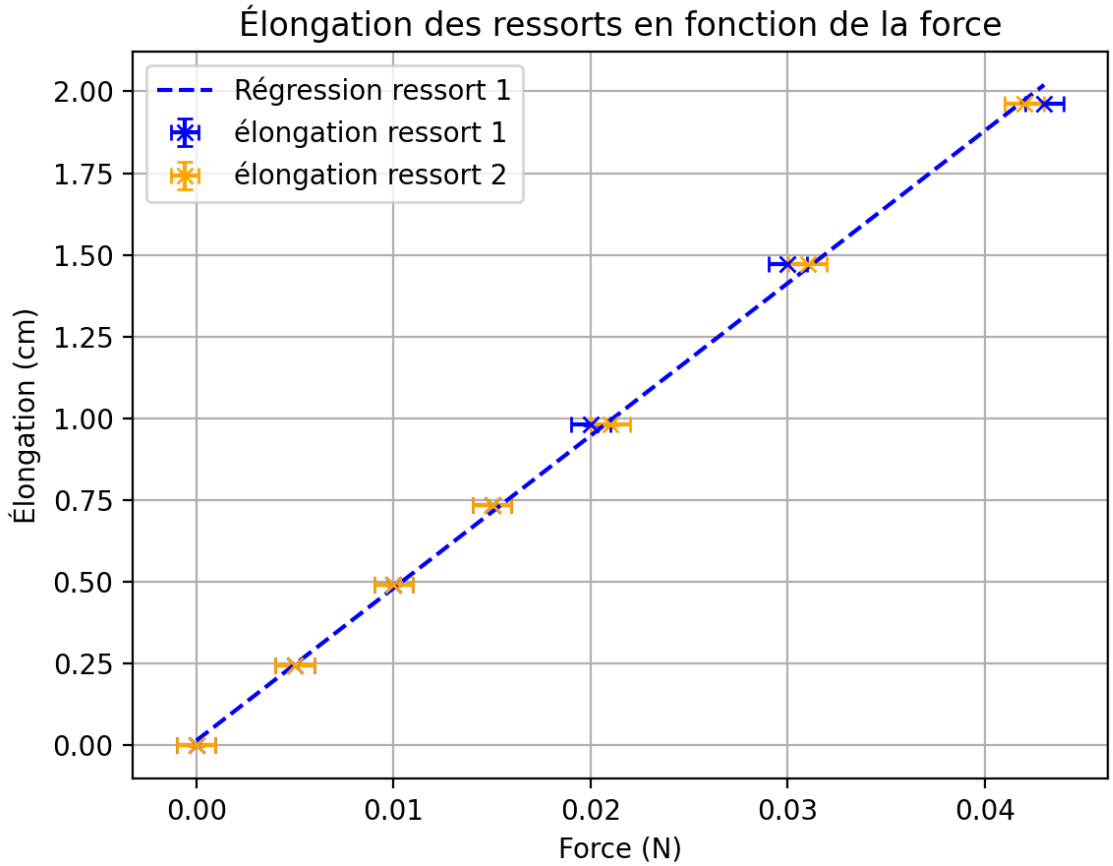
$$\begin{cases} N = mg\cos(\alpha) \\ T = mg\sin(\alpha) \end{cases}$$

Loi de Coulomb : $\left|\vec{T}\right| \leq f\left|\vec{N}\right|$

$$f = \tan(\alpha_{lim}) = \frac{H}{\sqrt{L^2 - H^2}}$$

Détermination constante raideur





PFD à l'équilibre :

$$0 = -mg + k(l - l_0)$$

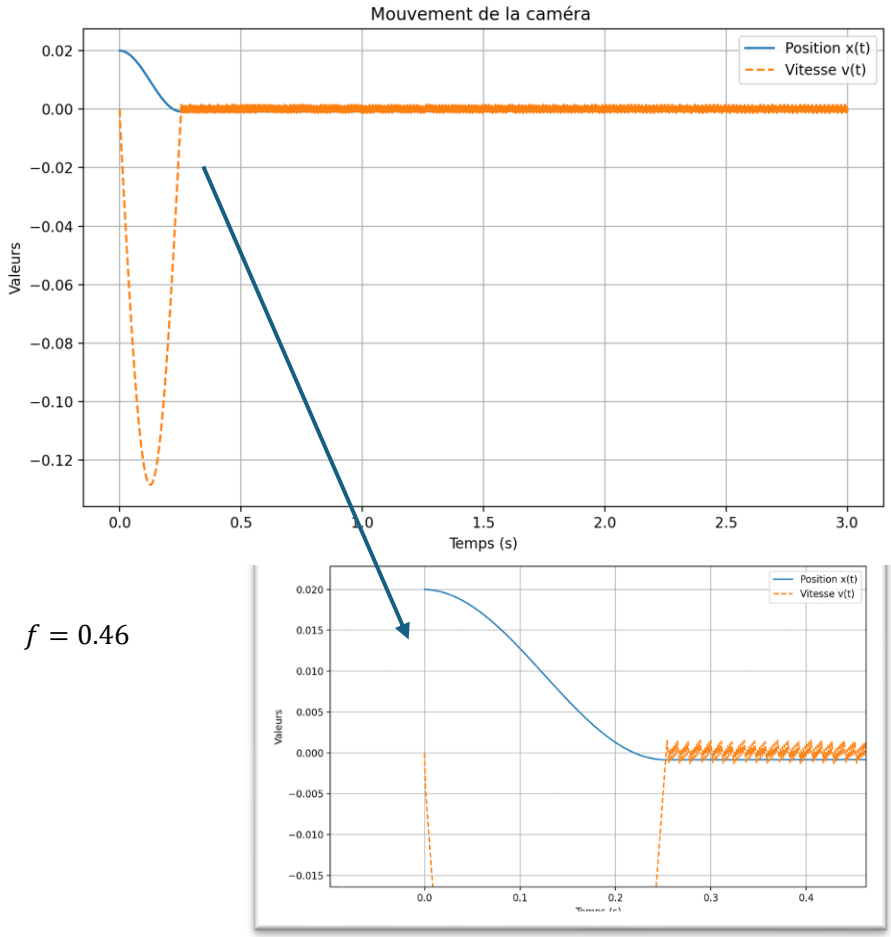
$$k = \frac{mg}{l - l_0}$$

$$u(m) = 0.1\ g \quad u(l) = u(l_0) = 1mm \quad u(g) = 0$$

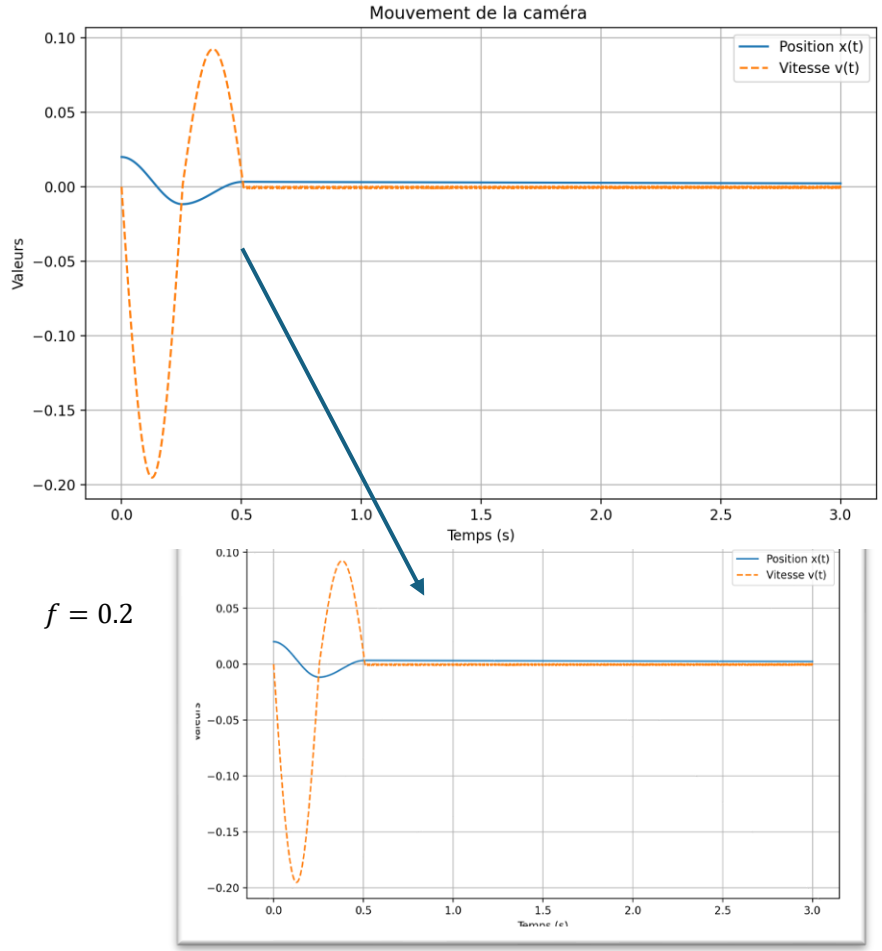
$$k_1 = 46.25\ N.m^{-1} \qquad k_2 = 46.62\ N.m^{-1}$$

$$k = 47.96 \pm 3.56\ N.m^{-1}$$

Simulation avec python



$f = 0.46$

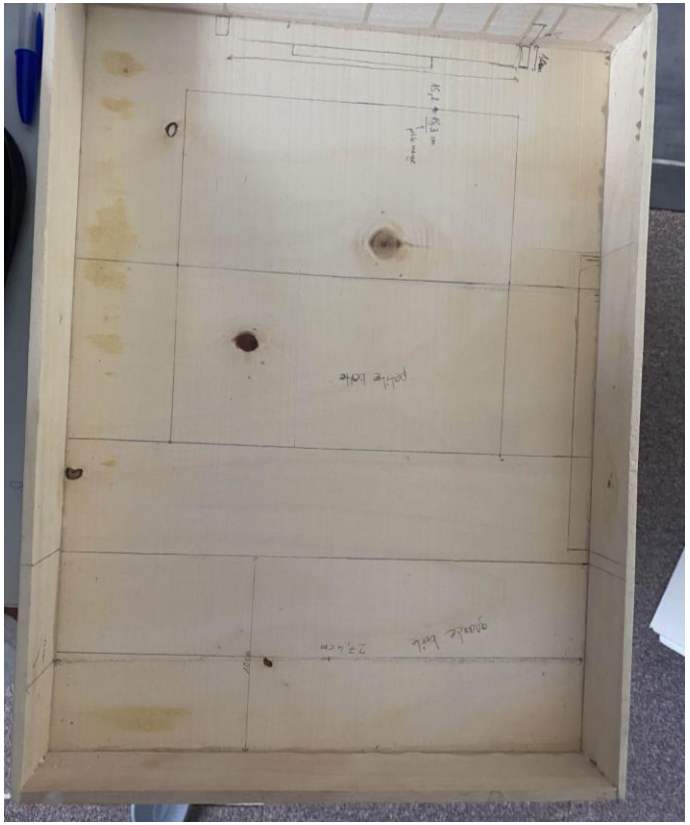


$f = 0.2$

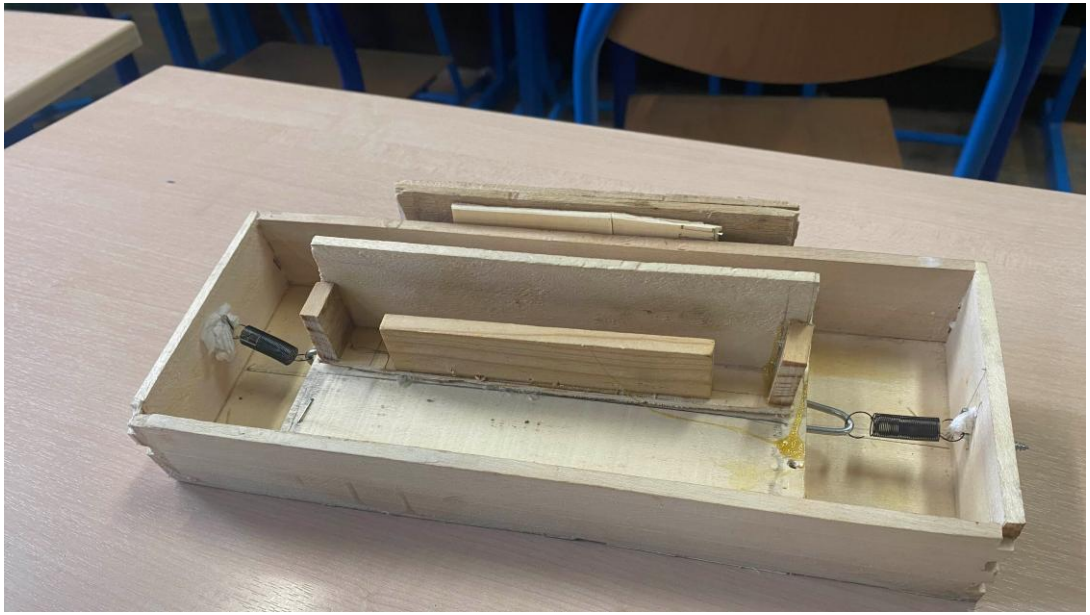
Masse petite boite : 97g
Masse téléphone + coque : 219g

Coefficient de frottements expérimentaux
0.46 ± 0.01 (bois-bois)
0.20 ± 0.03 (plastique-plastique)

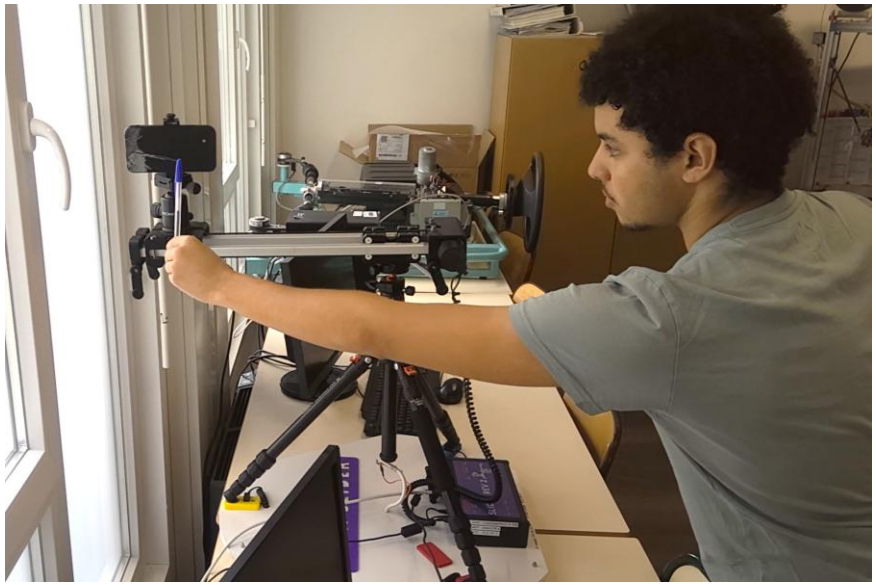
Constante de raideur : 48 $N.m^{-1}$

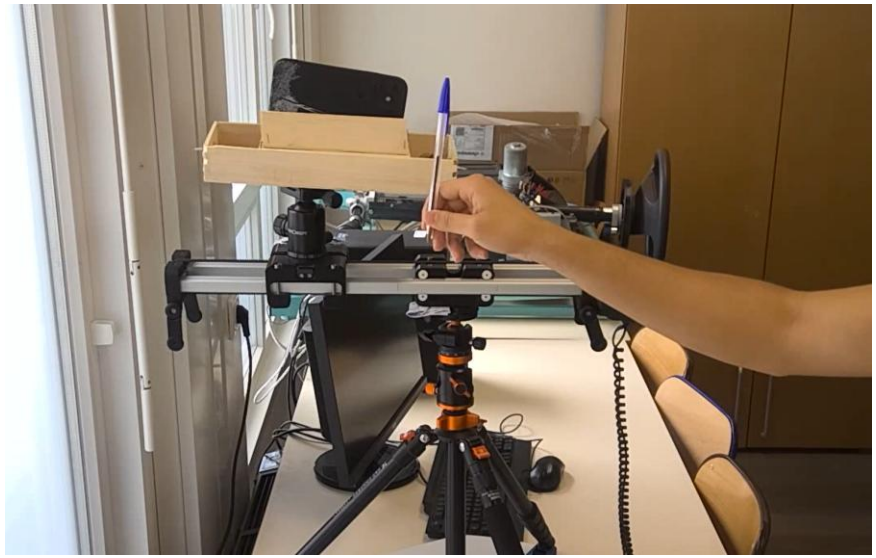


Conception et construction du stabiliseur

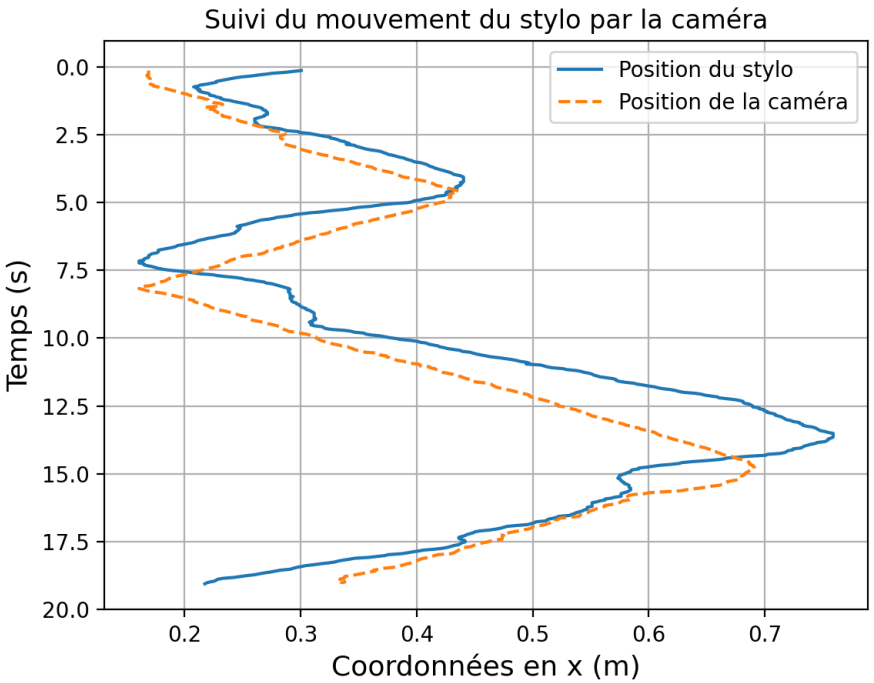


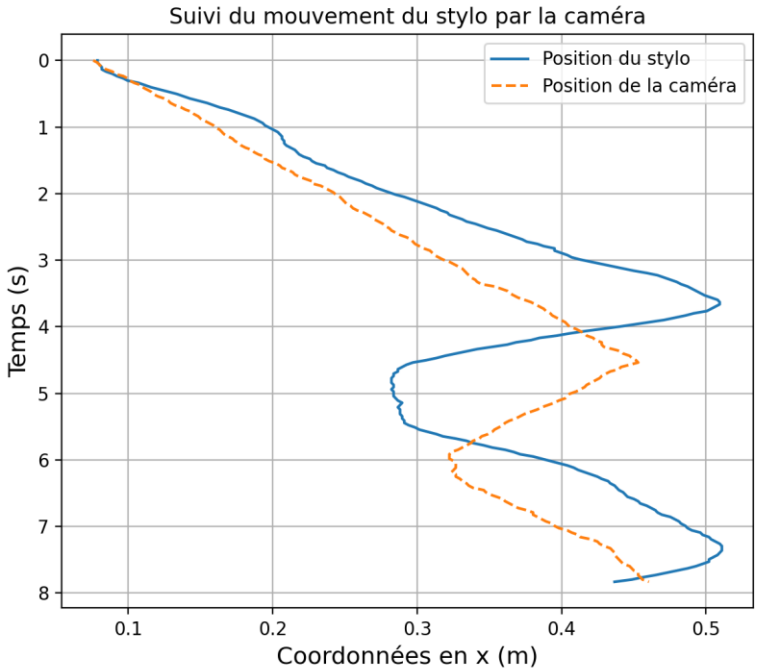
Expérience

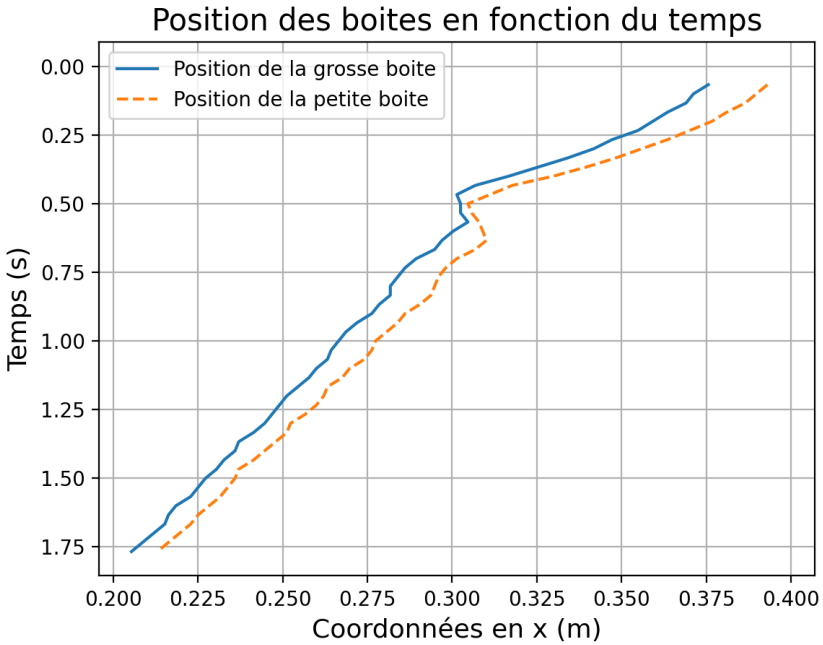




Expérience


Suivi du mouvement du stylo par la caméra
Position du stylo
Position de la caméra
Temps (s)
Coordonnées en x (m)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
0.2
0.3
0.4
0.5
0.6
0.7


Suivi du mouvement du stylo par la caméra
Position du stylo
Position de la caméra
Temps (s)
Coordonnées en x (m)
0
1
2
3
4
5
6
7
8
0.1
0.2
0.3
0.4
0.5


Position des boites en fonction du temps
Position de la grosse boite
Position de la petite boite
Temps (s)
Coordonnées en x (m)
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
0.200
0.225
0.250
0.275
0.300
0.325
0.350
0.375
0.400

Annexes

Les programmes de détection d’objet

Programme de détection de l'objet par couleur

```
import cv2

cap = cv2.VideoCapture(0)
# Set camera resolution
cap.set(3, 720)
cap.set(4, 480)
_, frame = cap.read()
rows, cols, _ = frame.shape
rows, cols, _ = frame.shape
center_x = int(cols/2)
center_y = int(rows/2)
acceptance_x = 50
acceptance_y = 20
x, y = center_x, center_y

while True:
    # Read the frame
    _, frame = cap.read()
    # Convert to gray
    grey_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Apply a tresholding

    l = 125
    h = 255
    _, mask = cv2.threshold(grey_frame, l, h,
cv2.ADAPTIVE_THRESH_GAUSSIAN_C)

    # Creation of the object's contours

    contours, _ = cv2.findContours(mask, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)
    for contour in contours:
        if cv2.contourArea(contour) > 1000:
            cv2.drawContours(frame, contour, -1, (0, 255, 0),
-1)
            # Calcul of the center of the object
            M = cv2.moments(contour)
            if M["m00"] != 0:
                x = int(M["m10"] / M["m00"])
                y = int(M["m01"] / M["m00"])
 # Draw a circle at the center of the object and write the
position
 cv2.circle(frame, (x, y), 7, (255, 255, 255), -1)

    contours, _ = cv2.findContours(mask, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)
    for contour in contours:
        if cv2.contourArea(contour) > 1000:
            cv2.drawContours(frame, contour, -1, (0, 255, 0), -1)
            # Calcul of the center of the object
            M = cv2.moments(contour)
            if M["m00"] != 0:
                x = int(M["m10"] / M["m00"])
                y = int(M["m01"] / M["m00"])
                # Draw a circle at the center of the object and
write the position
                cv2.circle(frame, (x, y), 7, (255, 255, 255), -1)


    cv2.imshow("Frame", frame)
    # Boucle exit condition
    key = cv2.waitKey(1)
    if key == 27: # press esc to exit
        break
    elif key == ord('s'):
            cv2.imwrite("saved_image.jpg", frame)
            cv2.imwrite("saved_blue_mask.jpg", mask)

cap.release()
cv2.destroyAllWindows()
```

Programme de détection de l'objet par contraste

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)
# Set camera resolution
cap.set(3, 720)
cap.set(4, 480)
_, frame = cap.read()
rows, cols, _ = frame.shape
center_x = int(cols/2)
center_y = int(rows/2)
acceptance_x = 50
acceptance_y = 20
x, y = center_x, center_y

colors = {"blue": [94, 80, 2, 126, 255, 255],
          "beige": [0, 64, 21, 116, 205, 128],
          "white": [0, 0, 255, 73, 101, 255],
          "pink": [126, 179, 167, 193, 255, 255],
          "yellow":  [0, 116, 206, 33, 198, 255],
          "light_green":  [31, 31, 172, 68, 198, 255],
          "orange": [7, 134, 54, 69, 255, 255],
          "black": [0, 0, 0, 255, 255, 10],
          "red": [2, 146, 76, 255, 255, 255]
         }
# Add the other colors
# They are one my personnal discord server

while True:
    try:
        color = str(input("Which color do you want to
track?")).lower().strip()
        np.array(colors[color][:3])
        break
    except:
        print("Choose a color among those one: blue, beige,
white, pink, yellow, light_green, orange, black")
```

```
while True:
    # Read the frame
    _, frame = cap.read()
    # Convert to HSV
    hsv_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

    low_color = np.array(colors[color][:3])
    high_color = np.array(colors[color][3:])

    # Create a mask
    mask = cv2.inRange(hsv_frame, low_color, high_color)

    # Creation of the object's contours



    # Display the frame
    cv2.imshow("Frame", frame)
    cv2.imshow("Mask", mask)
    # Boucle exit condition
    key = cv2.waitKey(1)
    if key == 27: # press esc to exit
        break
    elif key == ord('s'):
            cv2.imwrite("saved_image.jpg", frame)
            cv2.imwrite("saved_blue_mask.jpg", mask)

cap.release()
cv2.destroyAllWindows()
```

Programme de détection de l'objet à la main

```
import cv2

cap = cv2.VideoCapture(0)
# Set camera resolution
cap.set(3, 720)
cap.set(4, 480)
_, frame = cap.read()

rows, cols, _ = frame.shape
center_x = int(cols/2)
center_y = int(rows/2)
acceptance_x = 50
acceptance_y = 20
x, y = center_x, center_y
lower = 120
upper = 255

while True:
    # Read the frame
    _, frame = cap.read()
    # Convert to level of grey
    grey_frame = cv2.cvtColor(frame,
cv2.COLOR_BGR2GRAY)
    n, m = grey_frame.shape
    obj = []

    # filter grey to black and white
    for ligne in range(n):
        for colonne in range(m):
            if lower <=
grey_frame[ligne][colonne] <= upper:
                grey_frame[ligne][colonne] = 0
            else:
                grey_frame[ligne][colonne] =
255
                if ligne != n-1 and m-1 !=
colonne:
                    obj.append([ligne,
colonne])
```

```
    # Display the frame

    cv2.imshow("Frame", frame)
    cv2.imshow("Machine", grey_frame)
    key = cv2.waitKey(1)
    if key == 27:
        break
    elif key == ord('s'):
            cv2.imwrite("image.jpg", frame)


    # contour detection
    contours = []
    for i in range(len(obj)):
        ligne, colonne = obj[i][0], obj[i][1]
        if not grey_frame[ligne][colonne+1] ==
grey_frame[ligne][colonne-1] == grey_frame[ligne-1][colonne]
== grey_frame[ligne+1][colonne] == 255:
            contours.append((ligne, colonne))

    # centre des contours
    for i in range(len(contours)):
        y += contours[i][0]
        x += contours[i][1]
    x = x // n
    y = y // n
```

Programme de détection filtre Canny (1/3)

```
import cv2
import numpy as np

img = cv2.imread('1.png')

def Orientation(x, y):
    # on récupére l'angle de la direction du gradient
    if Gradient_x[x,y] == 0:
        theta = 90
        return theta
    theta = np.arctan(Gradient_y[x, y] / Gradient_x[x,
y])
    # on le compare avec des directions simples
    angle = [0, 45, 90, 135]
    dist = [abs(theta - angle[i]) for i in
range(len(angle))]
    index = dist.index(min(dist))
    # on retourne l'angle le plus proche
    theta = angle[index]
    return theta

# Convert to level of grey
grey_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
n, m = grey_img.shape

# apply gaussian blur
blurred = cv2.GaussianBlur(grey_img, (5, 5), 0)
```

```
# calcul of the gradient

Sx = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]])
Sy = np.array([[1, 2, 1], [0, 0, 0], [-1, -2, -1]])

Gradient_x = np.zeros((n, m))
Gradient_y = np.zeros((n, m))

# calcul of the approximation of the gradient
for i in range(1, n-1):
    for j in range(1, m-1):
        Gradient_x[i, j] = np.sum(Sx * grey_img[i-
1:i+2, j-1:j+2])
        Gradient_y[i, j] = np.sum(Sy * grey_img[i-
1:i+2, j-1:j+2])

Gradient = np.sqrt(Gradient_x**2 + Gradient_y**2)
```

Programme de détection filtre Canny (2/3)

```
# suppression of the noise

for i in range(1, n-1):
    for j in range(1, m-1):
        theta = Orientation(i, j)
        if theta == 0: #       0° (horizontal)
            if Gradient[i, j] < Gradient[i, j-1] or
Gradient[i, j] < Gradient[i, j+1]:
                Gradient[i, j] = 0
        elif theta == 45: #    45° (diagonale montante)
            if Gradient[i, j] < Gradient[i-1, j+1] or
Gradient[i, j] < Gradient[i+1, j-1]: # haut droite et bas
gauche
                Gradient[i, j] = 0
        elif theta == 90: #    90° (vertical)
            if Gradient[i, j] < Gradient[i-1, j] or
Gradient[i, j] < Gradient[i+1, j]:
                Gradient[i, j] = 0
        elif theta == 135: #   135° (diagonale descendante)
            if Gradient[i, j] < Gradient[i-1, j-1] or
Gradient[i, j] < Gradient[i+1, j+1]: # haut gauche et bas
droite
                Gradient[i, j] = 0
```

```
# hystreresis thresholding

# high threshold
high = 0.2 * np.max(Gradient)
# low threshold
low = 0.1 * np.max(Gradient)

def is_contour(x, y):
    # on regarde si les voisins de x, y sont des contours
    if Gradient[x-1, y] > high: #haut
        return True
    elif Gradient[x+1, y] > high: #bas
        return True
    elif Gradient[x, y-1] > high: #gauche
        return True
    elif Gradient[x, y+1] > high: #droite
        return True
    elif Gradient[x-1, y-1] > high: #haut gauche
        return True
    elif Gradient[x-1, y+1] > high: #haut droite
        return True
    elif Gradient[x+1, y+1] > high: #bas droite
        return True
    elif Gradient[x+1, y-1] > high: #bas gauche
        return True
    # si aucun voisin n'est un contour fort
    return False
```

Programme de détection filtre Canny (3/3)

```
# on parcourt l'image

for i in range(1, n-1):
    for j in range(1, m-1):
        if Gradient[i, j] > high:
            Gradient[i, j] = 255
        elif Gradient[i, j] < low:
            Gradient[i, j] = 0
        else:
            if is_contour(i, j):
                Gradient[i, j] = 255
            else:
                Gradient[i, j] = 0

# Display the frame

cv2.imshow('Image originale', img)
cv2.imshow('Contours détectés', Gradient )
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Programme de détection avec CNN

```
from tensorflow.keras import layers, models, Input

input_img = Input(shape=(640, 640, 3))

x = layers.Conv2D(32, (3,3), activation='relu')(input_img)
x = layers.MaxPooling2D(2,2)(x)
x = layers.Conv2D(64, (3,3), activation='relu')(x)
x = layers.MaxPooling2D(2,2)(x)
x = layers.Flatten()(x)
x = layers.Dense(128, activation='relu')(x)

# Sortie 1 : présence de l'objet
output_presence = layers.Dense(1, activation='sigmoid',
name='presence')(x)

# Sortie 2 : coordonnées de la bbox
output_bbox = layers.Dense(4, activation='sigmoid',
name='bbox')(x)  # sigmoid pour rester entre 0 et 1

model = models.Model(inputs=input_img,
outputs=[output_presence, output_bbox])
model.compile(
    optimizer='adam',
    loss={
        'presence': 'binary_crossentropy',
        'bbox': 'mean_squared_error'
    },
    metrics={
        'presence': 'accuracy',
        'bbox': 'mse'
    }
)
model.summary()
```

Les programmes de suivi

Fonctions pour le contrôle du slider

```python
from serial import *
import serial.tools.list_ports
import time

def swrite(ser, msg: str):
    # facilite l'écriture au slider
    ser.write("{}".format(msg).encode())


def sread(ser, n):
    # simplification de la lecture des réponses du slider
    return ser.read(n).decode("utf-8")


def rotation(ser, rot, angle):
    # simplification des commandes de rotation au slider
    # rot en rad/s
    swrite(ser, f"T{rot}/{angle}#")


def translation(ser, v, dist):
    # simplification des commandes de translation au slider
    # v en m/s
    swrite(ser, f"T{v}/{dist}#")

def pos_metre(ser):
    swrite(ser, "P#")
    posm = sread(ser, 5)
    return posm
```

```python
def pos_angle(ser):
    swrite(ser, "G#")
    posa = sread(ser, 5)
    return posa


def find_Slider():
    comms = serial.tools.list_ports.comports()
    for i in range(len(comms)):
        comms[i] = str(comms[i])
    for i in comms:
        if "USB" in i:
            comm = i.split("-")[0].strip()
            return True, comm
    return False, None

def origine(ser):
    swrite(ser, "O#")
    sread(ser, 2) # "retourne 'OK' "
```

Programme d’asservissement du slider (1/3)

```
from serial import *
import cv2
from math import sqrt
from time import sleep, time

from tracking2 import loc_obj
from commande_slider import *


cap = cv2.VideoCapture(1)
# Set camera resolution
cap.set(3, 720)
cap.set(4, 480)
_, frame = cap.read()
rows, cols, _ = frame.shape
center_x = int(cols/2)
center_y = int(rows/2)
acceptance_x = 50
acceptance_y = 20
x, y = center_x, center_y

# set slider
temps = 0.1 # en s ; c'est le temps entre de réaction du
slider
v_max = 0.125 # en m/s
rot_max = 9.3 # en rad/s
v = 0.1
d_max = 1.25
```

```
def distance_objet(pos_x, pos_y):
    dist_x = pos_x-center_x
    dist_y = pos_y-center_y
    distance = sqrt((dist_x)**2 + (dist_y)**2)
    return distance, dist_x, dist_y

def main():
    is_linked, slider_port = find_Slider()
    if is_linked: # si le slider est branché
        ser = Serial(port=slider_port, baudrate=115200,
timeout=1)
    else:
        print("Slider non branché")
        return None
    # on attend que la liaison se fait
    time.sleep(1)
    # on vérifie que le slider est opérationnel
    swrite(ser, "?#")
    if sread(ser, 2) != "OK":
        return

    # on se place à l'origine
    origine(ser)
    sleep(2)
    # On vide la ligne de réponse du slider, une fois à
l'origine, du cache
    sread(ser, 8)
```

Programme d’asservissement du slider (2/3)

```
    while True:
        try:
            filtre = input("Choose a filter : ").lower().strip()
            if filtre != "color filter" and filtre != 'grey
filter':
                raise TypeError
            break
        except TypeError:
            print("Choose a type of filter : 'color filter',
'grey filter'")
### Travail en cours
    pos = 0
    # boucle de suivi d'objet
    while True:
        start = now = time.time()
        _, frame = cap.read()
        pos_x, pos_y, obj_width, obj_height = loc_obj(cap,
filtre)
        # distance, dist_x, dist_y = distance_objet(pos_x,
pos_y)
        cv2.imshow("Frame", frame)

        # on se déplace pendant une durée à déterminer en
fonction de la réactivité du slider
```

```
        # Draw a rectangle at the center of the frame
        cv2.rectangle(frame, (center_x - acceptance_x,
center_y - acceptance_y), (center_x + acceptance_x, center_y
+ acceptance_y), (255, 255, 255), 2)
        cv2.circle(frame, (center_x, center_y), 7, (255,
255, 255), -1)

        #  !!!!!!!!!!!!! IT DEPENDS IF THE CAMERA GIVE A
REVERSED IMAGE OR NOT   !!!!!!!!!!!!
        # By default, I supposed that the camera gives a
image wich isn't reverse.

        dist = 0.01
        # Left and Right are only reversed graphically
        if pos_x > center_x + acceptance_x:
            cv2.putText(frame, "Right", (5, 20),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0), 2)
            if pos + dist < d_max:
                pos += dist
            translation(ser, v, pos)
        elif pos_x < center_x - acceptance_x:
            cv2.putText(frame, "Left", (5, 20),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 0), 2)
            if pos - dist >= 0:
                pos -= dist
            translation(ser, v, pos)
        else:
            translation(ser, v, pos)
```

Programme d’asservissement du slider (3/3)

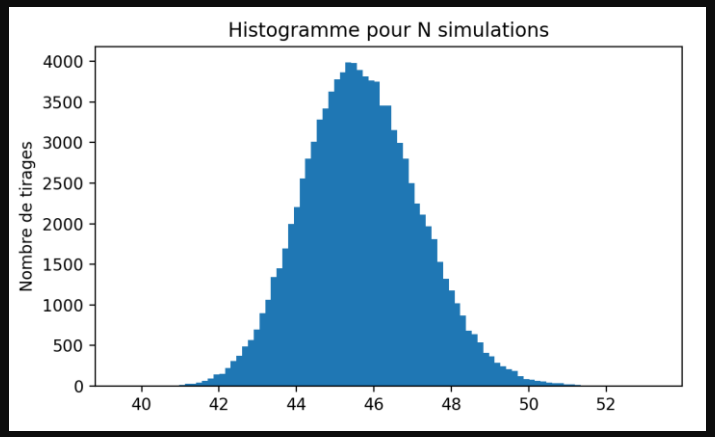
```
        temps = dist/v
        # affichage du flux vidéo pendant le déplacement
        while now - start <= temps:
            now = time.time()
            _, frame = cap.read()
            cv2.imshow("Frame", frame)



        key = cv2.waitKey(1)
        if key == 27: # echap -> on quitte
            break
        elif key == ord('s'):
                cv2.imwrite("saved_image.jpg", frame)

    # fermeture de la liaison série à la fin d'utilisation
    ser.close()


main()
```

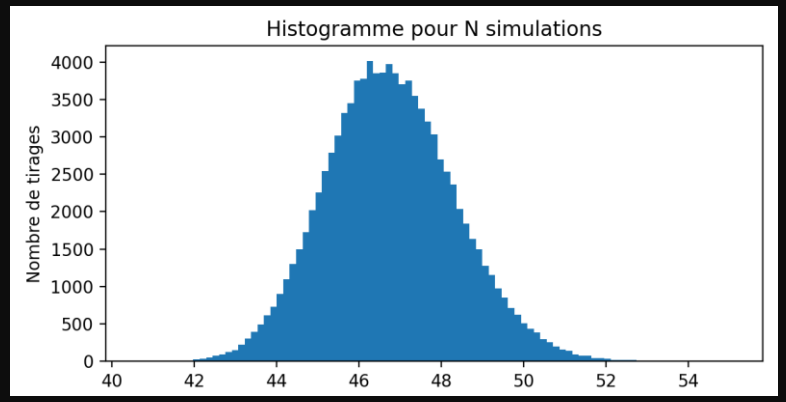
Les programmes calculs de grandeurs physiques

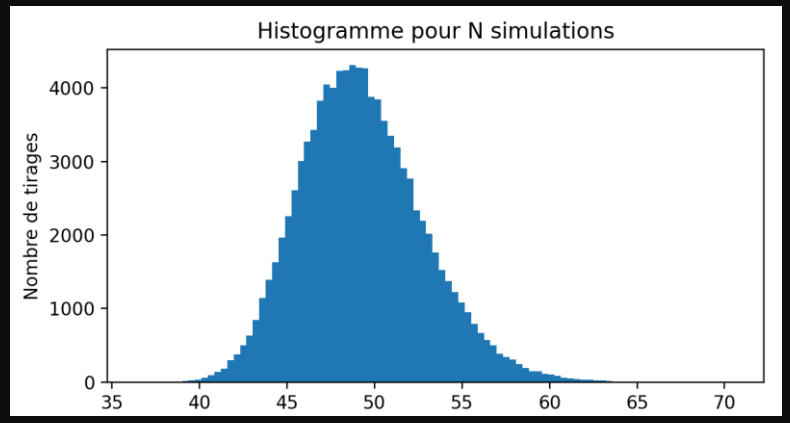
Programme calcul constante de raideur

Monte Carlo

```
import matplotlib.pyplot as plt
import numpy as np
import numpy.random as rd

N = 100000

l_0 = 2.7e-2 # en mètres
L = 7.0e-2 # en mètres
u_L = 1e-3 # en mètres
m = 200e-3 # en kilogramme
u_m = 1e-4 # en kilogramme
g  = 9.81

L_sim = rd.normal(L, u_L, N)
L_0_sim = rd.normal(l_0, u_L, N)
m_sim = rd.normal(m, u_m, N)

K_sim = (m_sim * g) / (L_sim - L_0_sim)
print(K_sim)

u_K = np.std(K_sim, ddof = 1)
K = np.mean(K_sim)
plt.rcParams["figure.figsize"] = (5,3)
fig, ax = plt.subplots()
plt.hist(K_sim, bins='rice')   #bins = nb de classes de
l'histogramme, rice = nb de classes optimisé
plt.title('Histogramme pour N simulations')
plt.xlabel('')
plt.ylabel('Nombre de tirages')
print('K = ' + str(K) + ' unite')
print('u(K) = ' + str(u_K) + ' unite')
plt.show()
```







Programme calcul constante de raideur régression linéaire

```python
import matplotlib.pyplot as plt
import numpy as np

# Données expérimentales
L1 = [2.7, 3.2, 3.7, 4.2, 4.7, 5.7, 7]
L2 = [2.7, 3.2, 3.7, 4.2, 4.8, 5.8, 6.9]
m = [0, 25e-3, 50e-3, 75e-3, 100e-3, 150e-3, 200e-3]
g  = 9.81

# Conversion en mètres
L1 = np.array(L1) * 1e-2
L2 = np.array(L2) * 1e-2
m = np.array(m)

# Incertitudes
delta_L = 0.1e-2  # 0.1 cm sur les longueurs
delta_m = 0.1e-3  # 0.1 g sur les masses

# Calcul des grandeurs utiles
F = m * g
elong1 = L1 - L1[0]
elong2 = L2 - L2[0]

# Incertitudes sur F et élongations
delta_F = delta_m * g
delta_elong = delta_L

# Tracé avec barres d'erreur
plt.errorbar(elong1, F, xerr=delta_F, yerr=delta_elong, fmt='x',
label='élongation ressort 1', color='blue', capsize=3)
plt.errorbar(elong2, F, xerr=delta_F, yerr=delta_elong, fmt='x',
label='élongation ressort 2', color='orange', capsize=3)
```

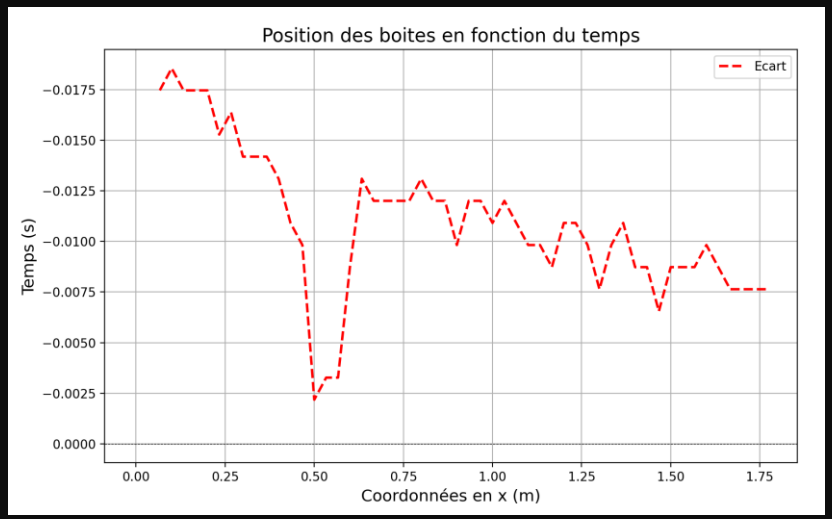
```python
a, b = np.polyfit(elong1, F, 1)
print(a, b)

a,b = np.polyfit(elong2, F, 1)
print(a, b)

plt.plot(elong1, a * elong1 + b, label='Régression ressort 1',
color='blue', linestyle='--')

# Mise en forme du graphique
plt.xlabel("Force (N)")
plt.ylabel("Élongation (m)")
plt.title("Élongation des ressorts en fonction de la force")
plt.legend()
plt.grid(True)
plt.show()
```

Programme de simulation mouvement stabilisateur

```python
import numpy as np
import matplotlib.pyplot as plt

# Paramètres du système
m = 0.316     # masse (kg)
k = 0.48     # raideur du ressort (N/m)
F_f = 0.4   # force de frottement sec (N)
x0 = 0.03    # position initiale (m)
v0 = 0.0     # vitesse initiale (m/s)

# Paramètres de la simulation
dt = 0.001                  # pas de temps (s)
t_max = 3.0                # durée totale (s)
N = int(t_max / dt)         # nombre d'itérations
t = np.linspace(0, t_max, N)

# Initialisation des vecteurs
x = np.zeros(N)
v = np.zeros(N)
a = np.zeros(N)

# Conditions initiales
x[0] = x0
v[0] = v0

# Fonction signe
def sgn(v):
    if v > 0:
        return 1
    elif v < 0:
        return -1
    else:
        return 0
```

```python
# Intégration numérique par méthode d'Euler explicite
for i in range(1, N):
    # Calcul de la force de frottement (seulement si la vitesse n'est pas nulle)
    if abs(v[i-1]) > 1e-8:
        f_frott = -F_f * sgn(v[i-1])
    elif abs(k * x[i-1]) > F_f: # relance le mouvement si la force de rappel dépasse le frottement
        f_frott = -F_f * sgn(k * x[i-1])
    else:
        f_frott = -k * x[i-1]  # équilibre statique, pas de frottement sec (bloqué)

    # Accélération
    a[i] = (-k * x[i-1] + f_frott) / m

    # Intégration (Euler)
    v[i] = v[i-1] + a[i] * dt
    x[i] = x[i-1] + v[i] * dt

# Affichage
plt.figure(figsize=(10, 6))
plt.plot(t, x, label='Position x(t)')
plt.plot(t, v, label='Vitesse v(t)', linestyle='--')
plt.xlabel('Temps (s)')
plt.ylabel('Valeurs')
plt.title('Oscillateur avec frottement sec')
plt.legend()
plt.grid(True)
plt.show()
```

Programme d'affichage de courbe

```python
from matplotlib import pyplot as plt
import numpy as np
import csv

with open("mvt_grosse_boite.csv", "r") as fichier:
    donnees = list(csv.reader(fichier, delimiter=';'))
    donnees.pop(0)

mvt  = []
time = []
for line in donnees:
    line = [i.replace(',', '.') for i in line]
    time.append(float(line[0]))
    mvt.append(float(line[1]))
with open("mvt_petite_boite.csv", "r") as fichier:
    donnees = list(csv.reader(fichier, delimiter=';'))
    donnees.pop(0)

mvt_cam  = []
time_cam = []
for line in donnees:
    line = [i.replace(',', '.') for i in line]
    time_cam.append(float(line[0]))
    mvt_cam.append(float(line[1]))
```

```python
plt.plot(mvt, time,  label="Position de la grosse boite")

plt.plot(mvt_cam, time_cam , label="Position de la petite boite", linestyle='--')
plt.gca().invert_yaxis()  # Invert the y-axis
plt.xlabel("Coordonnées en x (m)", fontsize=13)
plt.ylabel("Temps (s)", fontsize=13)
plt.title("Position des boites en fonction du temps", fontsize=15)
plt.legend()
plt.grid()
plt.show()
```